

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение  
высшего образования «Национальный исследовательский ядерный  
университет «МИФИ»

Кафедра №42 «Криптология и кибербезопасность»

Отчет о выпускной квалификационной  
работе бакалавра

по теме:  
СИСТЕМА МУЛЬТИФАКТОРНОЙ АУТЕНТИФИКАЦИИ

Студент:  
Группа Б22-515

А. С. Тимин

Научный руководитель:

К.Т.Н., доцент, в.н.с.  
Е. А. Басыня

## РЕФЕРАТ

Отчет 80 страниц, 39 иллюстраций, 4 таблицы, 37 источников, 4 приложения.

АУТЕНТИФИКАЦИЯ, АУТЕНТИФИКАТОР, ФАКТОР АУТЕНТИФИКАЦИИ, МУЛЬТИФАКТОРНАЯ АУТЕНТИФИКАЦИЯ, БАЗА ДАННЫХ АУТЕНТИФИКАЦИИ, СЕРВЕР АУТЕНТИФИКАЦИИ, АУТЕНТИФИЦИРУЮЩИЙ ТОКЕН, ОТР, БИОМЕТРИЯ, JWT, ТОКЕН

Объект исследования: информационные ресурсы вычислительной сети.  
Предмет исследования: методы мультифакторной аутентификации пользователя для доступа к информационным ресурсам вычислительной сети.

Цель работы: обеспечение аутентичности пользователей информационных ресурсов вычислительной сети посредством мультифакторной аутентификации.

Для достижения цели работы поставлены следующие задачи:

- 1) исследование предметной области;
- 2) моделирование объектов и процессов предметной области;
- 3) проектирование системы мультифакторной аутентификации;
- 4) программная реализация спроектированной системы;
- 5) тестирование и рефакторинг реализованной системы.

К основным результатам работы относятся:

- 1) сводная таблица результатов сравнительного анализа характеристик существующих систем мультифакторной аутентификации;
- 2) блок-схемы алгоритмов работы и UML-диаграммы последовательностей процессов моделей систем аутентификации;

- 3) UML-диаграмма компонентов, BPMN-диаграмма основного сценария работы, ER-диаграмма базы данных и блок-схемы алгоритмов работы спроектированной системы мультифакторной аутентификации;
- 4) сводная таблица технологического стека спроектированной системы мультифакторной аутентификации;
- 5) UML-диаграммы компонентов и их классов разработанной системы мультифакторной аутентификации;
- 6) сводные таблицы результатов произведенного тестирования разработанной системы мультифакторной аутентификации: интеграционного, нагрузочного и комплексного;
- 7) исходный код компонентов системы;
- 8) описание инфраструктуры виртуального лабораторного стенда с детальной инструкцией проведения автоматизированного тестирования разработанной системы мультифакторной аутентификации.

## СОДЕРЖАНИЕ

|                                                                          |           |
|--------------------------------------------------------------------------|-----------|
| <b>СОДЕРЖАНИЕ.....</b>                                                   | <b>4</b>  |
| <b>ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ.....</b>                        | <b>6</b>  |
| <b>ВВЕДЕНИЕ.....</b>                                                     | <b>10</b> |
| <b>1 ИССЛЕДОВАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ.....</b>                            | <b>13</b> |
| 1.1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ АУТЕНТИФИКАЦИИ .....                            | 13        |
| 1.2 СРАВНИТЕЛЬНЫЙ АНАЛИЗ СУЩЕСТВУЮЩИХ MFA-СИСТЕМ .....                   | 14        |
| 1.3 ИТОГИ.....                                                           | 17        |
| <b>2 МОДЕЛИРОВАНИЕ ОБЪЕКТОВ И ПРОЦЕССОВ ПРЕДМЕТНОЙ<br/>ОБЛАСТИ.....</b>  | <b>18</b> |
| 2.1 ОБЩАЯ МОДЕЛЬ АУТЕНТИФИКАЦИИ.....                                     | 18        |
| 2.2 МОДЕЛЬ АУТЕНТИФИКАЦИИ НА ОСНОВЕ ОТП .....                            | 20        |
| 2.3 МОДЕЛЬ БИОМЕТРИЧЕСКОЙ АУТЕНТИФИКАЦИИ .....                           | 22        |
| 2.4 ИТОГИ.....                                                           | 23        |
| <b>3 ПРОЕКТИРОВАНИЕ СИСТЕМЫ МУЛЬТИФАКТОРНОЙ<br/>АУТЕНТИФИКАЦИИ .....</b> | <b>24</b> |
| 3.1 ТРЕБОВАНИЯ К СИСТЕМЕ .....                                           | 24        |
| 3.2 КОМПОНЕНТЫ СИСТЕМЫ.....                                              | 25        |
| 3.3 ПРОЦЕССЫ СИСТЕМЫ .....                                               | 28        |
| 3.3.1 ОТП-токен .....                                                    | 30        |
| 3.3.2 Сервер аутентификации .....                                        | 32        |
| 3.4 ТЕХНОЛОГИЧЕСКИЙ СТЕК.....                                            | 39        |
| 3.5 ИТОГИ.....                                                           | 40        |
| <b>4 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ СПРОЕКТИРОВАННОЙ<br/>СИСТЕМЫ .....</b>       | <b>41</b> |
| 4.1 БД АУТЕНТИФИКАЦИИ.....                                               | 41        |
| 4.2 СЕРВЕР АУТЕНТИФИКАЦИИ .....                                          | 43        |
| 4.2.1 Развертывание .....                                                | 43        |
| 4.2.2 Компоненты и классы.....                                           | 46        |
| 4.3 ОТП-ТОКЕН.....                                                       | 51        |
| 4.4 ИТОГИ.....                                                           | 53        |
| <b>5 ТЕСТИРОВАНИЕ И РЕФАКТОРИНГ РЕАЛИЗОВАННОЙ<br/>СИСТЕМЫ .....</b>      | <b>55</b> |
| 5.1 ВИРТУАЛЬНАЯ ЛАБОРАТОРИЯ.....                                         | 55        |
| 5.2 ИНТЕГРАЦИОННОЕ ТЕСТИРОВАНИЕ .....                                    | 56        |
| 5.3 НАГРУЗОЧНОЕ ТЕСТИРОВАНИЕ .....                                       | 60        |

|                                                                                             |           |
|---------------------------------------------------------------------------------------------|-----------|
| 5.4 КОМПЛЕКСНОЕ ТЕСТИРОВАНИЕ.....                                                           | 64        |
| 5.5 ИТОГИ.....                                                                              | 67        |
| <b>ЗАКЛЮЧЕНИЕ .....</b>                                                                     | <b>69</b> |
| <b>СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ .....</b>                                               | <b>71</b> |
| <b>ПРИЛОЖЕНИЕ А. ССЫЛКА НА РАЗРАБОТАННОЕ РЕШЕНИЕ .....</b>                                  | <b>76</b> |
| <b>ПРИЛОЖЕНИЕ Б.<br/>ССЫЛКА НА ВИДЕОДЕМОНСТРАЦИЮ РАБОТЫ РАЗРАБОТАННОГО<br/>РЕШЕНИЯ.....</b> | <b>77</b> |
| <b>ПРИЛОЖЕНИЕ В. ФАЙЛ LOCUST_..._REQUESTS.CSV.....</b>                                      | <b>78</b> |
| <b>ПРИЛОЖЕНИЕ Г. ФАЙЛ LOCUST_..._FAILURES.CSV .....</b>                                     | <b>80</b> |

## ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

Информационная безопасность (ИБ) – комплекс мер и средств, направленных на обеспечение конфиденциальности, целостности и доступности информации.

Конфиденциальность информации – аспект информационной безопасности, определяющий необходимость предотвращения разглашения или утечки информации.

Целостность информации – аспект информационной безопасности, определяющий необходимость неизменности информации нелегитимным субъектом.

Доступность информации – аспект информационной безопасности, определяющий необходимость беспрепятственной реализации прав на информацию легитимными субъектами.

Вычислительная сеть – система, обеспечивающая обмен данными между вычислительными устройствами (компьютерами, серверами и другим оборудованием) или программным обеспечением.

Аутентичность – свойство некоторого субъекта, позволяющее установить, что он действительно является тем, за кого или за что себя выдаёт.

Идентификация (identification) – процесс выявления отличительного признака (идентификатора) пользователя вычислительной сети.

Аутентификация (authentication) – процесс верификации идентификационных данных, заявленных пользователем вычислительной сети.

Авторизация (authorization) – процесс предоставления пользователю соответствующих привилегий в вычислительной сети.

Субъект доступа (в контексте аутентификации) – конкретный человек или процесс, проходящий аутентификацию.

Объект доступа (в контексте аутентификации) – информационный ресурс или сервис, для доступа к которому необходимо пройти аутентификацию.

Идентификатор (в контексте аутентификации) – опознавательный знак, отличающий данного субъекта доступа от остальных.

Аутентификатор – отличительная характеристика, подтверждающая принадлежность идентификатора субъекту доступа.

Фактор аутентификации – определенный вид аутентификатора.

Однофакторная аутентификация (1FA – One Factor Authentication) – аутентификация на основе одного фактора аутентификации.

Мультифакторная аутентификация (MFA – Multi-Factor Authentication) – аутентификация на основе двух и более факторов аутентификации.

ОТР (One-Time Password) – пароль однократного использования (одноразовый).

ОТР-токен – персональное устройство, предназначенное для генерации ОТР.

JWT (JSON Web Token) – открытый стандарт (RFC 7519), определяющий компактный и автономный способ передачи информации между сторонами в формате JSON, который может быть аутентифицирован благодаря цифровой подписи.

БД (DB – Data Base) – база данных.

ПО – программное обеспечение.

Интерфейс программирования приложения (API – Application Programming Interface) – контракт, описывающий взаимодействия между программными сущностями, определяющий допустимые операции, их сигнатуры и способы передачи данных.

Микросервис – небольшой, слабосвязный, независимо развертываемый сервис, реализующий одну бизнес-функцию и взаимодействующий с другими сервисами посредством легковесных протоколов.

Микросервисная архитектура – архитектурный стиль, при котором ПО строится как набор микросервисов, каждый из которых выполняется в собственном процессе, управляется автономной командой, развертывается и масштабируется независимо от остальных, взаимодействует с ними посредством предоставляемых API.

Человек посередине (MitM – Man in the Middle) – вид атаки на компьютерную сеть, при которой злоумышленник внедряется в канал передачи данных, получая возможность перехватывать, ретранслировать и искажать передаваемую по нему информацию.

UML (Unified Modeling Language) – унифицированный язык моделирования.

ERD (Entity-Relationship Diagram) – диаграмма сущность-связь, представляющая логическую структуру базы данных в виде графического изображения.

BPMN (Business Process Model and Notation) – язык моделирования бизнес-процессов.

ЯП – язык программирования.

ООП (объектно-ориентированное программирование) – парадигма программирования, при которой программа рассматривается как набор взаимодействующих друг с другом объектов.

ORM (Object-Relational Mapping) – технология, позволяющая разработчикам манипулировать данными в базе данных, используя возможности ООП вместо написания SQL-запросов.

Слоистая архитектура – структурный шаблон, при котором система разбивается на иерархические уровни (слои), каждый из которых решает конкретную подзадачу и взаимодействует только с соседними, что обеспечивает заменяемость компонентов.

Интеграционное тестирование – уровень тестирования ПО, на котором отдельные модули или компоненты объединяются и тестируются, как группа для выявления дефектов их взаимодействия.

Нагрузочное тестирование – вид тестирования, предназначенный для оценки поведения и производительности при заданной ожидаемой нагрузке на ПО.

Комплексное тестирование – уровень тестирования ПО, на котором полностью интегрированная система проверяется на соответствие установленным требованиям.

## **ВВЕДЕНИЕ**

Актуальность темы исследования определяется современными вызовами в области информационной безопасности. В условиях цифровизации общества, когда значительная часть данных передается и обрабатывается в информационных системах, защита информации становится критически важной задачей. Одной из основных угроз остается несанкционированный доступ к конфиденциальным данным.

Согласно Указу Президента РФ №899 от 07.07.2011 года «Об утверждении приоритетных направлений развития науки, технологий и техники в Российской Федерации», информационная безопасность входит в перечень критических технологий, требующих активного развития. Одним из приоритетных направлений является внедрение надежных механизмов аутентификации, способных противостоять актуальным угрозам.

По данным «Лаборатории Касперского», в 2023 году в России было зафиксировано 656 киберинцидентов, связанных с утечкой персональных данных, при этом было похищено приблизительно 1,12 млрд записей. Для сравнения, в 2022 году число инцидентов составляло 770, а количество скомпрометированных записей – около 702 млн.

Аналитики отмечают, что в 2023 году подавляющее большинство утечек персональных данных в РФ произошло в результате кибератак (82,7%), 10,1% случаев связаны с умышленными действиями внутренних сотрудников предприятий, а 1,2% – со случайными действиями работников.

Внедрение мультифакторной аутентификации является одним из эффективных способов повышения уровня безопасности информационных систем. По прогнозам аналитиков MTC RED, объем российского рынка MFA к 2027 году вырастет на 80% и составит 5,6 млрд рублей.

Таким образом, тема исследования является актуальной, поскольку направлена на разработку системного решения, способного обеспечить высокий уровень безопасности информационных ресурсов.

Цель настоящей работы – обеспечение аутентичности пользователей информационных ресурсов вычислительной сети посредством мультифакторной аутентификации.

Для достижения поставленной цели работы были определены 5 основных задач, каждой из которых посвящена соответствующая глава отчета:

- 1) исследование предметной области;
- 2) моделирование объектов и процессов предметной области;
- 3) проектирование системы мультифакторной аутентификации;
- 4) программная реализация спроектированной системы;
- 5) тестирование и рефакторинг реализованной системы.

К основным результатам выполненной работы относятся:

- 1) сводная таблица результатов сравнительного анализа характеристик существующих систем мультифакторной аутентификации;
- 2) блок-схемы алгоритмов работы и UML-диаграммы последовательностей процессов моделей систем аутентификации;
- 3) UML-диаграмма компонентов, BPMN-диаграмма основного сценария работы, ER-диаграмма базы данных и блок-схемы алгоритмов работы спроектированной системы мультифакторной аутентификации;
- 4) сводная таблица технологического стека спроектированной системы мультифакторной аутентификации;
- 5) UML-диаграммы компонентов и их классов разработанной системы мультифакторной аутентификации;

6) сводные таблицы результатов произведенного тестирования разработанной системы мультифакторной аутентификации: интеграционного, нагрузочного и комплексного;

7) исходный код компонентов системы;

8) описание инфраструктуры виртуального лабораторного стенда с детальной инструкцией проведения автоматизированного тестирования разработанной системы мультифакторной аутентификации.

## 1 Исследование предметной области

В начале разработки любой системы необходимо подробно изучить предмет исследования, а также проанализировать существующие на рынке альтернативы на предмет определения их преимуществ и недостатков.

В данной главе будут рассмотрены теоретические основы аутентификации и проведен сравнительный анализ предоставляемого функционала популярных систем мультифакторной аутентификации.

### 1.1 Теоретические основы аутентификации

В современном мире многие сервисы прибегают к *персонализации данных* – вычислительные сети собирают, обрабатывают и хранят различную пользовательскую информацию, для чего необходимо однозначно соотносить данные с их владельцем.

Решением описанной проблемы является *регистрация* пользователя, в рамках которой для него создается *учетная запись*, определяемая уникальным *идентификатором* (далее – ID), подтверждаемым с помощью *аутентификатора* (далее – PW). В дальнейшем вся личная информация привязывается к этому аккаунту. Теперь для доступа к приватным данным пользователю сервиса необходимо проходить три этапа: *идентификацию*, *аутентификацию* и *авторизацию*.

Простейший алгоритм аутентификации состоит из трех шагов:

- 1) *субъект доступа* предоставляет свой PW;
- 2) сервер верифицирует полученный PW с эталоном для данного пользователя;
- 3) результат проверки отправляется пользователю.

В настоящее время существует классификация РW по *факторам*: *знание* (пароль, PIN-код и прочие), *владение* (физический ключ, ОTR-токен и прочие) и *свойство* (отпечаток пальца, рисунок радужки глаза, клавиатурный почерк и прочие). По комбинации различных факторов выделяют *однофакторную* и *мультифакторную* (многофакторную) аутентификации. Применение последней представляет дополнительный слой защиты, значительно повышая издержки компрометации пользовательских данных.

## 1.2 Сравнительный анализ существующих MFA-систем

С приходом осознания необходимости использования систем мультифакторной аутентификации различные компании принялись разрабатывать и предоставлять свои решения в этой сфере. К примеру, высокую популярность имеют продукты компаний Яндекс и Google.

Двухфакторная аутентификация в Яндекс.Паспорт комбинирует факторы свойства и владения. Для ее прохождения пользователь заходит в мобильное приложение по отпечатку пальца, после чего сканирует QR-код, предоставляемый специальным сервисом. Мобильное приложение выступает в роли ОTR-токена, генерирующего 8-значные одноразовые пароли, которые вводятся пользователем в соответствующее поле в клиентском ПО. Также для доступа к ОTR-токену в Яндексе предусмотрели возможность использования фактора знания вместо биометрического посредством ввода ранее установленного PIN-кода. Такое решение понижает уровень защищенности, однако является необходимой компромиссной мерой для повышения доступности системы (если на устройстве пользователя отсутствует сканер отпечатка пальца).

Мобильное приложение Authenticator от Google нельзя в полной мере назвать MFA-системой. Этот инструмент реализует лишь дополнительный фактор аутентификации в подспорье основному. Мобильное приложение

представляет собой OTP-токен, генерирующий временно действующие одноразовые 6-значные числовые пароли. В отличие от системы Яндекса решение Google легко встраивается во сторонние сервисы за счет того, что его серверная часть на различных языках программирования опубликована в открытом доступе.

Также помимо вышеописанных существуют и другие системы многофакторной аутентификации:

- «Multifactor»;
- «Indeed Access Manager (AM)»;
- «Аладдин Р.Д. JaCarta Authentication Server (JAS)»;
- «Avanpost FAM и Web SSO»;
- и другие.

Для проведения объективного сравнения необходимо заранее определиться с критериями, в соответствие с которыми оно будет произведено. В статье [20] приводится набор различных критериев, некоторые из которых представляют большую значимость для системы MFA:

- *комбинация факторов.* В зависимости от используемого количества и набора факторов будет присвоено значение формата d-d, где d – число, обозначающее номер фактора: 1 – знание, 2 – владение, 3 – свойство;
- *направление аутентификации.* Одностороннее (аутентификацию проходит только субъект доступа) или двустороннее (аутентификацию также проходит сервер);
- *криптография аутентификатора.* Применение механизмов шифрования или хэширования при передаче аутентификационных данных.

Также помимо описанных критериев необходимо добавить:

- *встраиваемость*. Возможность встраивания системы MFA в стороннюю инфраструктуру в качестве микросервиса (или комбинации микросервисов);
- *кроссплатформенность*. Возможность работы системы MFA на различных аппаратных платформах или ОС.

Выбранные критерии позволят провести простейшее сравнение характеристик систем мультифакторной аутентификации для выявления их преимуществ и недостатков.

На основании выбранных критериев было проведено сравнение MFA-систем компаний Яндекс и Google, результаты сведены в таблицу 1.1.

Таблица 1.1 – Сводная таблица результатов сравнительного анализа MFA-систем

| Критерий сравнения           | «Яндекс.Паспорт» | «Authenticator» |
|------------------------------|------------------|-----------------|
| Комбинация факторов          | 2-3 или 1-2      | 2-              |
| Направление аутентификации   | Односторонняя    | Односторонняя   |
| Криптография аутентификатора | Хэширование      | Хэширование     |
| Встраиваемость               | -                | +               |
| Кроссплатформенность         | +                | +               |

Как видно из таблицы, важных различий между представленными системами всего два: комбинация факторов и встраиваемость. В случае Authenticator использование только одного фактора аутентификации не является проблемой, так как данная система обычно используется в комбинации с другими системами защиты, усиливая их. А вот невозможность встраивания Яндекс.Паспорт в стороннее ПО делает ее неприменимой вне сервисов Яндекса.

Также следует заметить, что обе системы не используют двустороннюю аутентификацию, что является потенциальной уязвимостью к MitM-атакам.

### 1.3 Итоги

В данной главе был рассмотрен теоретический базис об аутентификации, а также проанализированы популярные реализации MFA-систем.

По итогу исследования предметной области выявлено, что рассмотренные реализации потенциально имеют в себе серьезную уязвимость (подмена аутентифицирующего сервера посредством MitM-атаки), в то время как нивелирование данной угрозы не является сложно реализуемой задачей. Также недостатком упомянутых систем является реализация лишь двухфакторной аутентификации.

Целью данной работы является обеспечение аутентичности пользователей информационных ресурсов вычислительной сети посредством мультифакторной аутентификации. Способ достижения поставленной цели – разработка MFA-системы, которая помимо прочего будет нивелировать вышеописанную угрозу, а также комбинировать в себе все три фактора аутентификации, что положительно скажется на уровне ее защищенности.

## 2 Моделирование объектов и процессов предметной области

Этапу проектирования программного решения зачастую предшествует моделирование объектов и процессов предметной области, в рамках которого анализируются типовые модели функционирования ПО без относительно конкретных технологий их реализации.

В данной главе будут представлены и проанализированы блок-схемы алгоритмов работы и UML-диаграммы последовательностей процессов моделей систем аутентификации.

### 2.1 Общая модель аутентификации

На рисунках 2.1 и 2.2 изображены в обобщенном виде блок-схема алгоритма и UML-диаграмма последовательности аутентификации.

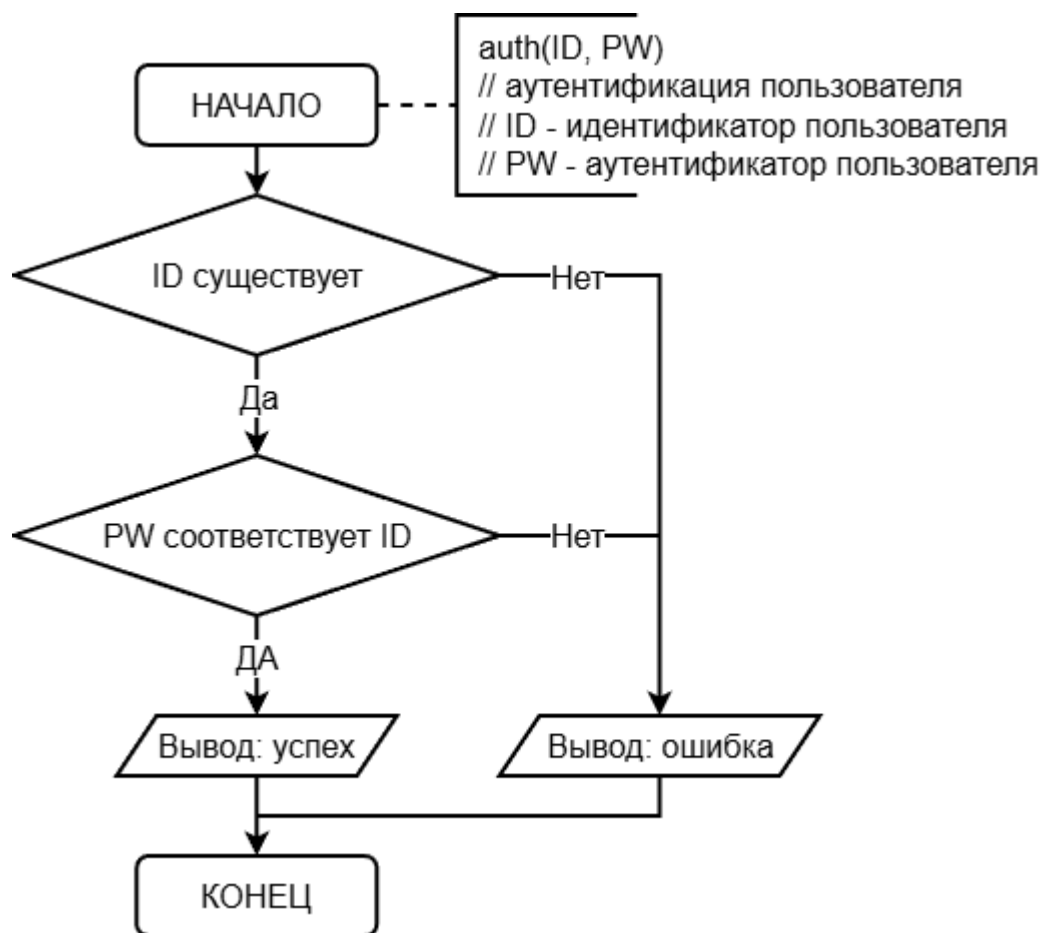


Рисунок 2.1 – блок-схема обобщенного алгоритма аутентификации

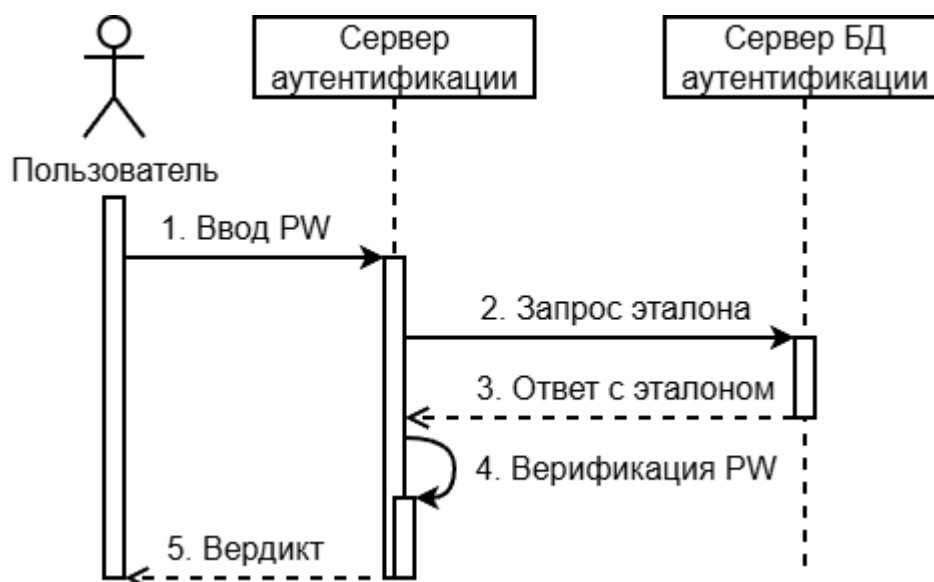


Рисунок 2.2 – UML-диаграмма обобщенной последовательности аутентификации

Для краткости на схемах использовано обозначение PW – сокращение от password (пароль), однако под данным обозначением следует понимать любой аутентификатор, используемый в конкретной системе.

Схема аутентификации по фактору знания зачастую полностью соответствует представленным рисункам.

Проследим простейший пример аутентификации по фактору знания. Человек (назовем его Боб), выступая в роли субъекта доступа, хочет обратиться к сервису электронной почты, чтобы ознакомиться с новым письмом. При открытии почтового клиента Боб видит форму входа, представляющую собой набор из двух полей и кнопки отправки данных. В первое поле он вводит адрес своей электронной почты «bob.user@post.com», по совместительству являющийся его ID (при регистрации новой учетной записи почтовые сервисы следят за уникальностью выбранного адреса). Во второе поле Боб вводит пароль «b0B~siGm@», придуманный им на этапе создания аккаунта. В данном примере пароль выступает в роли PW, подтверждающего, что именно этот пользователь присутствовал при регистрации учетной записи, то есть он является ее владельцем (или доверенным лицом). После нажатии кнопки форма отправляется по сети на сервера почтового сервиса, где компонент

аутентификации сверяет введенные данные с эталонами в базе данных и выдает вердикт в зависимости от результата проверки. По итогам прошедшей аутентификации пользователь проходит авторизацию, в рамках которой почтовый сервис либо предоставляет Бобу доступ к письму, либо сообщает о нехватке прав (в случае неудачной аутентификации).

## 2.2 Модель аутентификации на основе OTP

Рассмотренный в разделе 2.1 пример аутентификации относится к аутентификации по фактору знания – пользователь вынужден помнить свой пароль. Такой способ аутентификации не слишком надежен, так как субъект доступа может попросту забыть свой PW, из-за чего он потеряет доступ к своей учетной записи.

Одно из решений описанной проблемы – использование аутентификации по фактору владения. Здесь на помощь приходит персональный аутентифицирующий токен, в роли которого могут выступать: физические ключи, смарт-карты, USB-ключи, специальные приложения и т.д.

Заострим внимание на применение OTP-токенов. По своему режиму работы они делятся на *синхронные* и *асинхронные* (рисунки 2.3 и 2.4). Различаются они следующим образом: синхронный токен генерирует одноразовые пароли на основе *скрытого вопроса* (например, определенная на этапе регистрации операция с временной меткой или инициализирующими данными), в свою очередь асинхронный токен вычисляет OTP на основе *challenge* (обычно, случайных данных, полученных от сервера).

В случае обоих режимов на сервер аутентификации возлагается задача верификации полученного OTP, для чего серверу необходимо вычислить эталонное значение одноразового пароля или извлечь из него данные, помещенные токеном (например, зашифрованное число, равное удвоенному значению challenge).

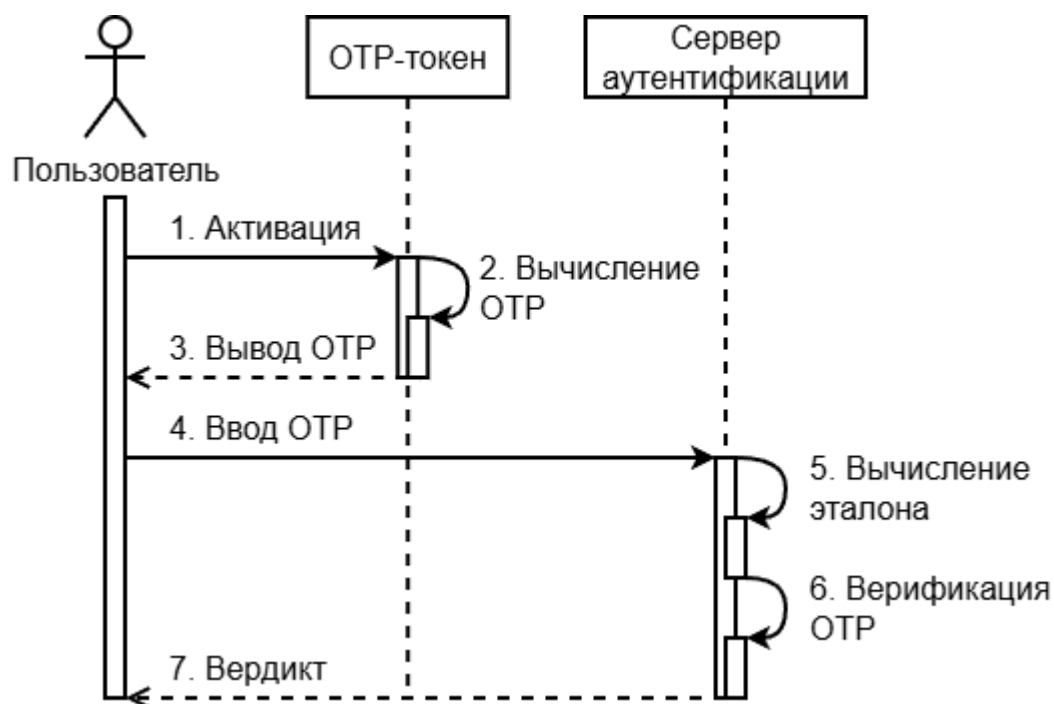


Рисунок 2.3 – UML-диаграмма последовательности синхронной OTP-аутентификации

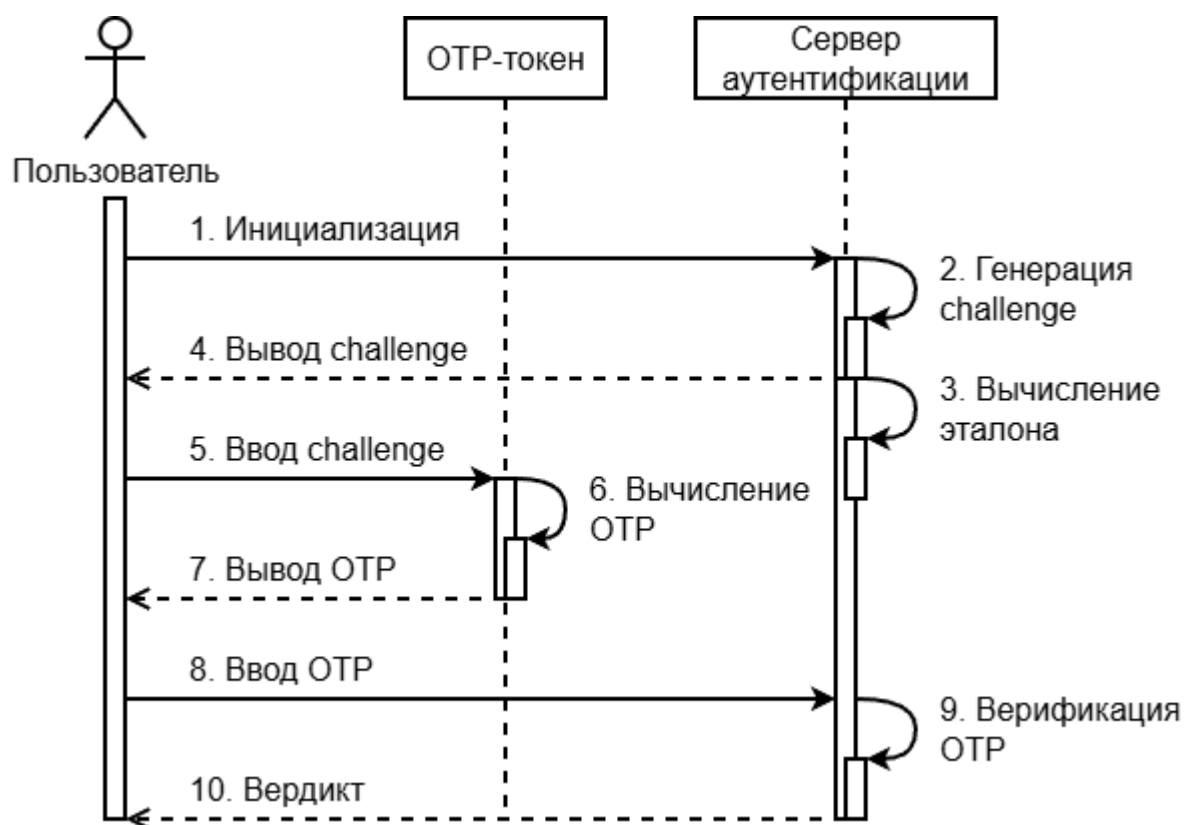


Рисунок 2.4 – UML-диаграмма последовательности асинхронной OTP-аутентификации

## 2.3 Модель биометрической аутентификации

Аутентификация на основе фактора владения имеет ряд недостатков, среди которых утеря аутентифицирующего токена, создание его дубликата злоумышленником и необходимость всегда иметь при себе этот токен.

Аутентификация на основе фактора свойства (биометрическая аутентификация) полностью избавляет от описанных проблем. Принято разделять ее на аутентификацию по физиологическим (отпечаток пальца, рисунок радужки глаз и другие) и поведенческим (динамическим; клавиатурный почерк, подпись и другие) признакам. Такое разделение носит весьма условный характер, так как многие динамические признаки в значительной мере зависят от физиологических (подпись – от ловкости и размеров пальцев).

Если говорить о схемах биометрической аутентификации, то они в точности повторяют обобщенные (рисунки 2.1 и 2.2) с той лишь особенностью, что пользователь не сам вводит PW, а использует специализированный датчик (например, сканер отпечатка пальца, фотокамеру или микрофон).

Однако биометрическая аутентификация не лишена и недостатков. В частности, из-за сложности сканирования PW (пользователь прикладывает палец не всегда в одинаковом положении, из-за болезни голос может быть изменен и т.д.) невозможно добиться абсолютного соответствия – применяется оценка достоверности: если совпадение выше установленного порогового значения, то аутентификация считается успешной. Очевидно, что при использовании подобных алгоритмов вероятно возникновение ложноположительных и ложноотрицательных сработок, соотношение которых представляет собой основную метрику оценки эффективности конкретной системы биометрической аутентификации. Тем не менее, удобство применения для пользователя во многих случаях превышает описанные недостатки.

## **2.4 Итоги**

В данной главе были рассмотрены и проанализированы модели систем аутентификации для факторов знания, обладания и свойства. Каждая из моделей имеет свои преимущества и недостатки. Комбинации моделей, образующие системы мультифакторной аутентификации, на основе разных факторов способны повысить преимущества и нивелировать недостатки отдельно каждой из них. Особенно ценно, что модели MFA-систем значительно повышают защищенность, так как даже в случае взлома одного фактора злоумышленник не получает доступ к учетной записи пользователя.

Многообразие методов аутентификации создает огромный выбор для их внедрения в различное программное обеспечение. В каждом отдельном случае разработчик решает, какие из систем подтверждения личности наилучшим образом встроятся в его продукт и будут обеспечивать надежный уровень защиты при минимизации финансовых вложений.

### **3 Проектирование системы мультифакторной аутентификации**

Одним из важнейших этапов разработки какого-либо программного комплекса является проектирование. В данной главе как раз приводятся результаты проектирования разрабатываемой системы мультифакторной аутентификации.

#### **3.1 Требования к системе**

Для начала, следует определиться с общими требованиями к проектируемой системе. Сделать это можно на основании результатов сравнительного анализа существующих на рынке популярных MFA-систем, проведенного ранее в рамках исследования предметной области (раздел 1.2), а также выявления преимуществ и недостатков рассмотренных методов аутентификации (глава 2). Чтобы нивелировать часть описанных недостатков следует предъявить следующие требования к проектируемой системе:

- 1) двусторонняя аутентификация;
- 2) встраиваемость системы в стороннюю инфраструктуру;
- 3) комбинирование трех факторов аутентификации;
- 4) передача аутентификаторов по сети в зашифрованном виде;
- 5) аутентификация владельца ОТР-токена при его активации;
- 6) аутентификация по событию (счетчик успешных входов);
- 7) аутентификация по времени (ограниченный период действия ОТР);
- 8) прогрессивная шкала временной задержки после неудачной попытки входа или активации ОТР-токена;

9) уничтожение секретного ключа и блокировка учетной записи пользователя при достижении лимита неудачных попыток активации ОТР-токена или входа в систему.

Применение трехфакторной аутентификации (пароль, ОТР и отпечаток пальца) превращают систему с точки зрения потенциального злоумышленника в сложную цель, повышая издержки ее взлома. Двусторонняя аутентификация гарантирует, что злоумышленник не сможет выдать себя за сервер аутентификации. Шифрование аутентификаторов при передаче по сети делает невозможным их подмену при проведении MitM-атак. Обязательная аутентификация владельца при активации ОТР-токена позволяет избежать эксплуатацию утерянного или краденного токена злоумышленником. Применение аутентификации по событию и по времени минимизирует вероятность случайного угадывания ОТР. Временные задержки после неудачных попыток аутентификации и блокировки при достижении лимита неудач предотвращают bruteforce-атаки.

Таким образом, указанные требования позволят нивелировать часть потенциальных уязвимостей проектируемой системы мультифакторной аутентификации.

### **3.2 Компоненты системы**

На основании общей концепции системы мультифакторной аутентификации, а также предъявленных к ней требований определен набор ее компонентов, представленный на рисунке 3.1. Важно отметить, что изображенный на рисунке объект «Внешняя инфраструктура» напрямую не относится к разрабатываемой системе. Он является частью инфраструктуры того решения, куда впоследствии может быть встроена проектируемая MFA-система.

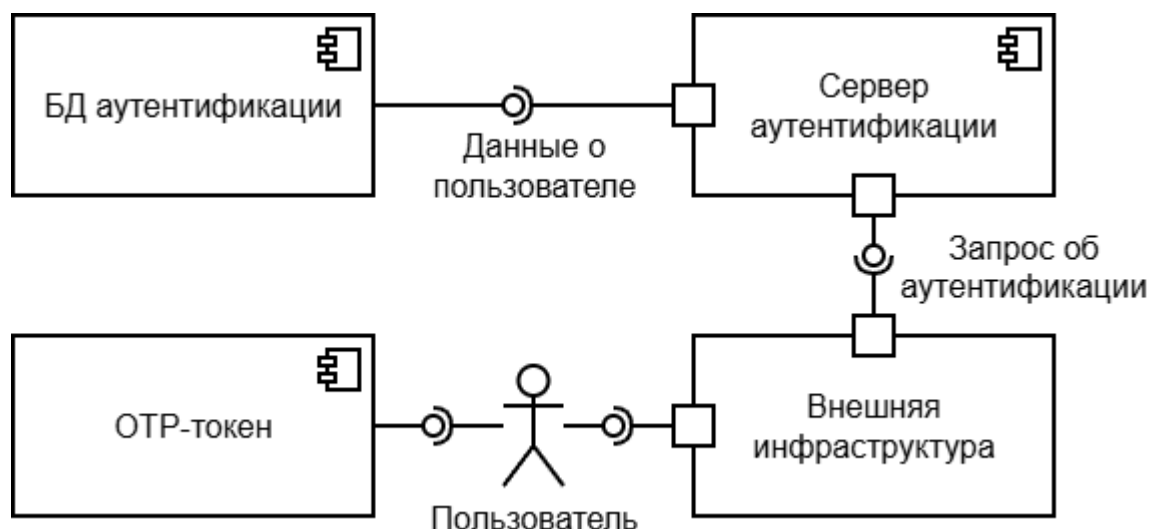


Рисунок 3.1 – UML-диаграмма компонентов проектируемой MFA-системы

Один из компонентов – реляционная база данных – хранит информацию, необходимую для аутентификации пользователя (рисунок 3.2), при этом каждому соответствует единственная запись в таблице users и неопределенное количество записей о refresh-токенах, необходимых для упрощенного прохождения повторной аутентификации (после полноценной трехфакторной аутентификация производится по refresh и access токенам). Рассмотрим подробнее назначение каждого поля.

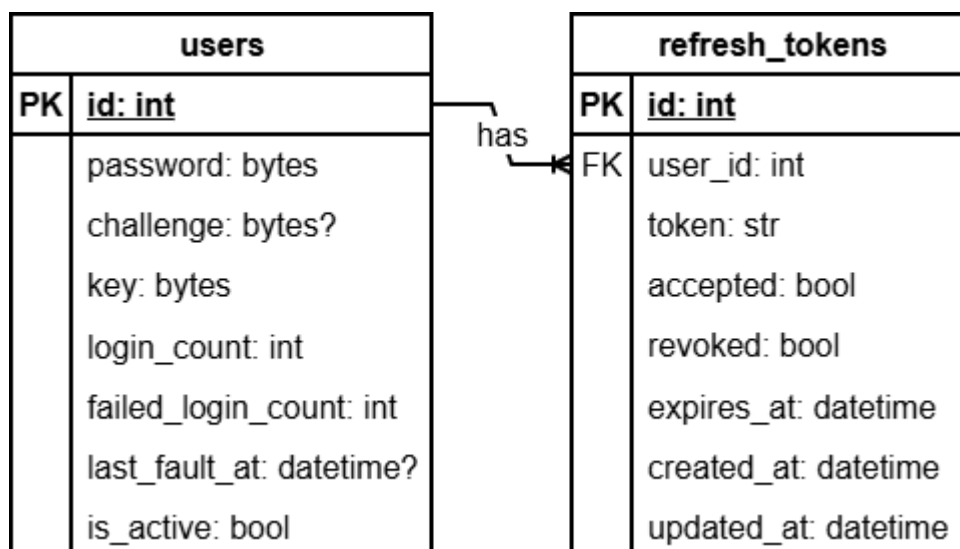


Рисунок 3.2 – ER-диаграмма БД аутентификации

Таблица users состоит из 8 колонок:

- 1) `id` – целое число, суррогатный первичный ключ, уникальный идентификатор пользователя;
- 2) `password` – комбинация соль+хэш пароля пользователя;
- 3) `challenge` – комбинация `random+timestamp`, зашифрованная на секретном ключе приложения, создаваемая для вычисления ОТР;
- 4) `key` – секретный ключ пользователя, зашифрованный на секретном ключе приложения, применяемый для шифрования и расшифрования `challenge` и ОТР;
- 5) `login_coins` – целое число, счетчик успешных входов, применяемый для генерации и верификации ОТР;
- 6) `failed_login_count` – целое число, счетчик неудачных входов, применяемый для отслеживания достижения лимита неудач;
- 7) `last_fault_at` – метка времени последней неудачи, применяемая для временной блокировки пользователя;
- 8) `is_active` – флаг подтвержденности учетной записи пользователя.

Таблица `refresh_tokens` также состоит из 8 колонок:

- 1) `id` – целое число, суррогатный первичный ключ, уникальный идентификатор refresh-токена;
- 2) `user_id` – целое число, внешний ключ, привязывающий refresh-токен к конкретной учетной записи пользователя;
- 3) `token` – строка, уникальное значение refresh-токена;
- 4) `accepted` – флаг подтвержденности refresh-токена (refresh-токен с поднятым флагом может быть использован для выпуска access-токенов);
- 5) `revoked` – флаг отозванности refresh-токена (отозванный refresh-токен не может использоваться для выпуска access-токенов);

- 6) expires\_at – временная метка, срок годности refresh-токена;
- 7) created\_at – метка времени создания записи о refresh-токене;
- 8) updated\_at метка времени изменения записи о refresh-токене.

Спроектированная схема базы данных удовлетворяет нормальным формам: 1НФ (все атрибуты атомарны), 2НФ (отсутствуют частичные зависимости), 3НФ (отсутствуют транзитивные зависимости) и BCNF (все детерминанты – суперключи).

### 3.3 Процессы системы

На основании предъявленных требований сформирована BPNM-диаграмма основного сценария взаимодействия пользователя с проектируемой MFA-системой – первичная аутентификация (рисунок 3.3). На рисунке схематично представлено взаимодействие пользователя с OTP-токеном и сервером аутентификации, а также последовательность выполняемых ими операций.

Пошагово проанализируем представленную диаграмму:

- 1) пользователь инициирует вход в систему, вводя идентификационные данные, которые отправляются на сервер аутентификации;
- 2) сервер аутентификации запрашивает из БД данные текущего пользователя, генерирует challenge и отправляет его пользователю;
- 3) также сервер аутентификации вычисляет эталонное значение OTP;
- 4) тем временем пользователь активирует токен, предоставляя ему необходимые аутентификаторы для подтверждения своей личности;
- 5) после чего пользователь вводит в токен challenge;



Представленная схема намеренно несколько упрощена. Реальная ее вариация не предполагает прямого взаимодействия сервера аутентификации и пользователя. Роль прослойки между ними будут выполнять компоненты той инфраструктуры, в которую будет встроена проектируемая MFA-система. Также на данной диаграмме не представлено подробного описания всех алгоритмов работы компонентов системы, а лишь общая схема взаимодействия с пользователем.

### 3.3.1 OTP-токен

OTP-токен представляет собой мобильное приложение на Android, необходимое для генерации одноразовых паролей на основе challenge сформированного сервером аутентификации. Токен имеет три основные функции: регистрация, активация и удаление данных. На рисунках 3.4–3.7 представлены блок-схемы этих алгоритмов.

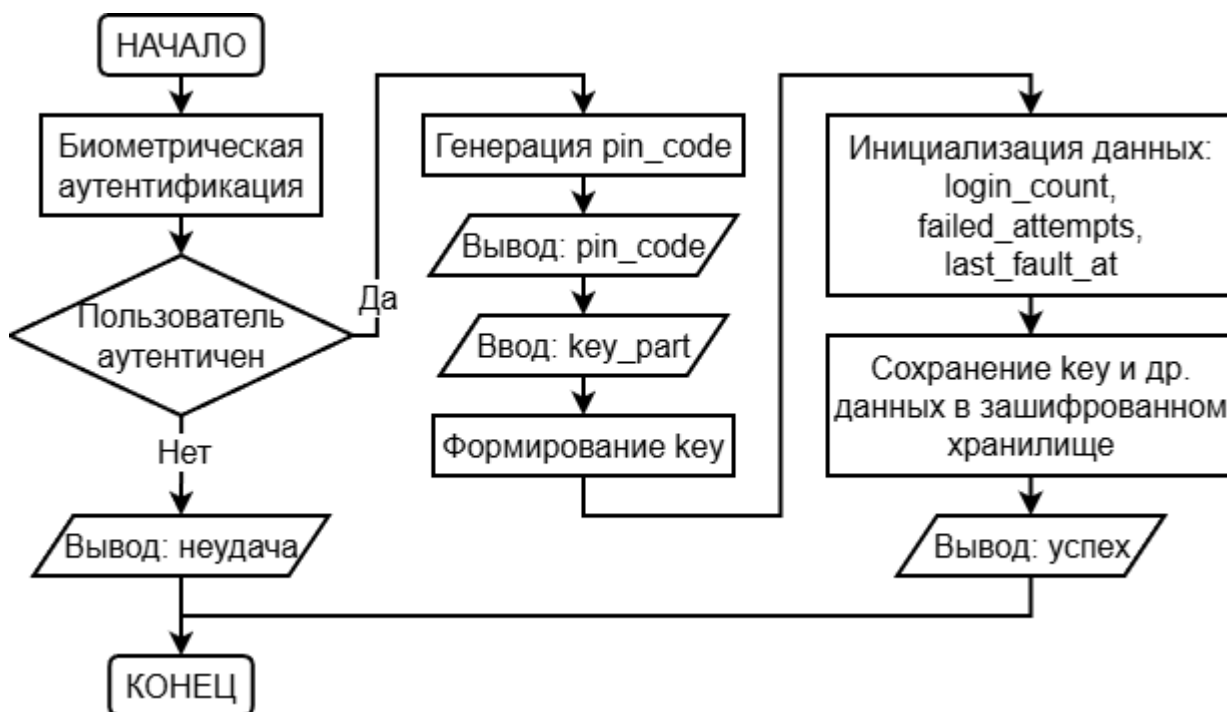


Рисунок 3.4 – блок-схема алгоритма регистрации OTP-токена

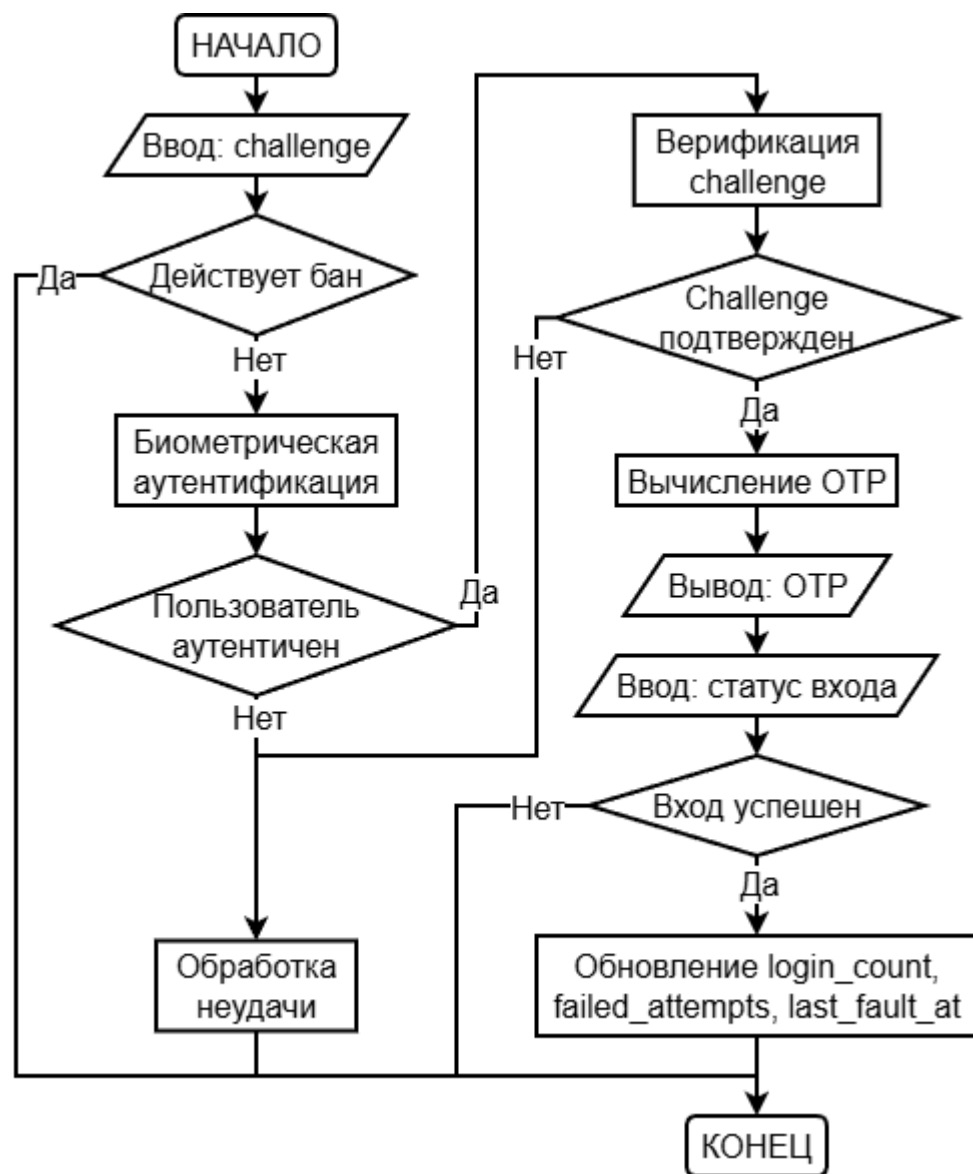


Рисунок 3.5 – блок-схема алгоритма активации OTP-токена



Рисунок 3.6 – блок-схема алгоритма обработки неудачи при активации OTP-токена

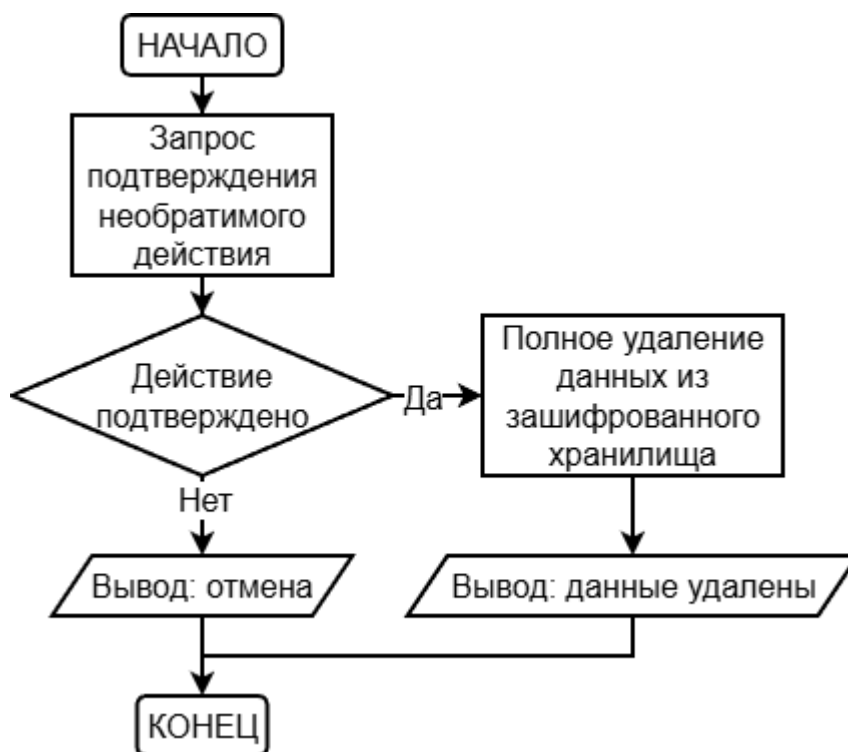


Рисунок 3.7 – блок-схема алгоритма сброса данных OTP-токена

В алгоритмах помимо прочих присутствует блок «Биометрическая аутентификация», выполнение которого предполагается осуществлять посредством вызова соответствующей функции через API ОС Android. Важно упомянуть, что данные в OTP-токене хранятся в зашифрованном на секретном ключе приложения виде.

Также стоит отметить, что счетчик успехов используется для генерации OTP и должен соответствовать ожидаемому на сервере значению. Именно поэтому важно синхронизировать OTP-токен после успешного входа на сервере.

### 3.3.2 Сервер аутентификации

Рассмотрим подробнее другой важный компонент проектируемой системы – сервер аутентификации. Как видно из диаграммы компонентов (рисунок 3.1), сервер аутентификации имеет два основных порта: для взаимодействия с БД и для взаимодействия с внешней инфраструктурой. Эти

порты активно используются в процессе работы его алгоритмов: регистрации нового пользователя, входа существующего пользователя, обновления access-токена, верификации access-токена и выхода пользователя.

На рисунках 3.8 и 3.9 представлены блок-схемы алгоритмов работы двухэтапной регистрации нового пользователя.

На этапе инициализации регистрации происходит создание учетной записи для нового пользователя и генерация фрагмента для составного ключа пользователя. Чтобы идентифицировать пользователя, проходящего регистрацию в данный момент, сервер выпускает для него специальный неподтвержденный короткоживущий refresh-токен.

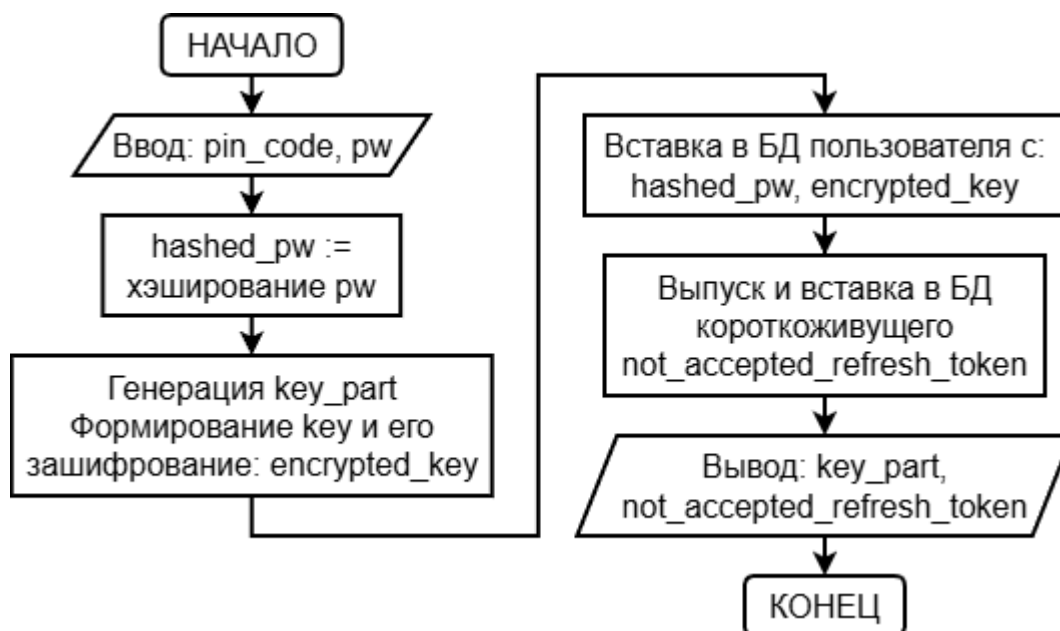


Рисунок 3.8 – блок-схема алгоритма инициализации регистрации пользователя на сервере

Далее, чтобы подтвердить свою регистрацию на сервере пользователю необходимо пройти ее второй этап. Для этого пользователь предоставляет полученный ранее refresh-токен. Далее сервер проверяет наличие в базе данных присланного токена и связанной учетной записи пользователя: токен должен быть неподтвержденным, неотозванным и непросроченным – в противном случае в регистрации будет отказано. Если данные корректны, сервер активирует учетную запись пользователя, отзывает старый токен и

выпускает пару новых: refresh и access – после чего возвращает пользователю. На этом пользователь считается зарегистрированным

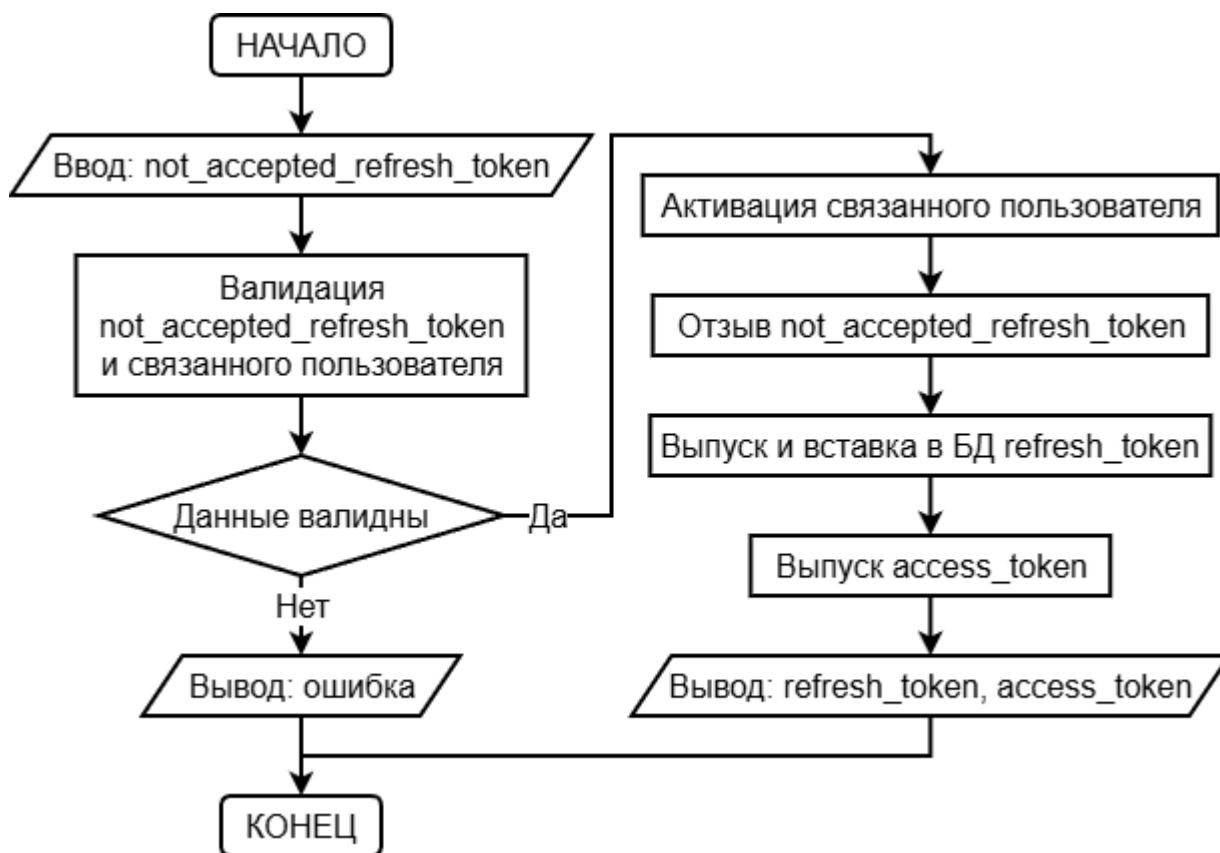


Рисунок 3.9 – блок схема алгоритма подтверждения регистрации пользователя на сервере

Уже несколько раз ранее был упомянут access-токен. Он представляет собой относительно короткоживущий JWT, хранящий помимо прочих данных ID пользователя. Токен подписан с помощью секретного ключа по асимметричному алгоритму шифрования. Для проверки подписи используется публичный ключ, свободно предоставляемый сервером аутентификации по соответствующему запросу.

Также сервер предоставляет возможность проверить валидность access-токена при отправке специального запроса. На рисунке 3.10 представлена блок-схема алгоритма работы соответствующей функции. Как видно из рисунка, в процессе проверки access-токена сервер не имеет необходимости обращаться к базе данных. Данная функция может активно использоваться

внешней инфраструктурой для быстрой проверки аутентичности пользователя без дополнительной нагрузки на базу данных.

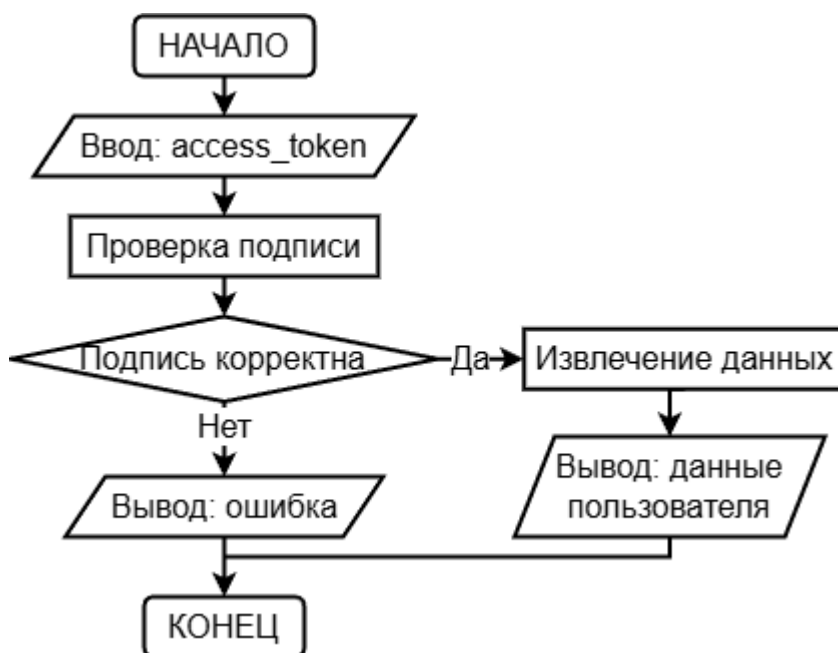


Рисунок 3.10 – блок-схема алгоритма верификации access-токена

Одной из особенностей access-токенов является их короткий срок жизни. Когда он истекает появляется необходимость выпуска нового токена. На этот случай сервер предоставляет возможность обновления JWT. На рисунке 3.11 представлена блок-схема алгоритма работы функции обновления access-токена.

В отличие от функции подтверждения регистрации обновление access-токена требует предоставление подтвержденного refresh-токена, а также чтобы связанная учетная запись была активирована. В случае успешной проверки данных сервер выпускает новый access-токен и отправляет пользователю.

В отличие от функции верификации access-токена данная функция выполняет запросы к базе данных. Тут становится понятным причина использования двух различных токенов – стремление снизить нагрузку на базу данных аутентификации, ведь для верификации JWT достаточно лишь алгоритм шифрования и ключ.

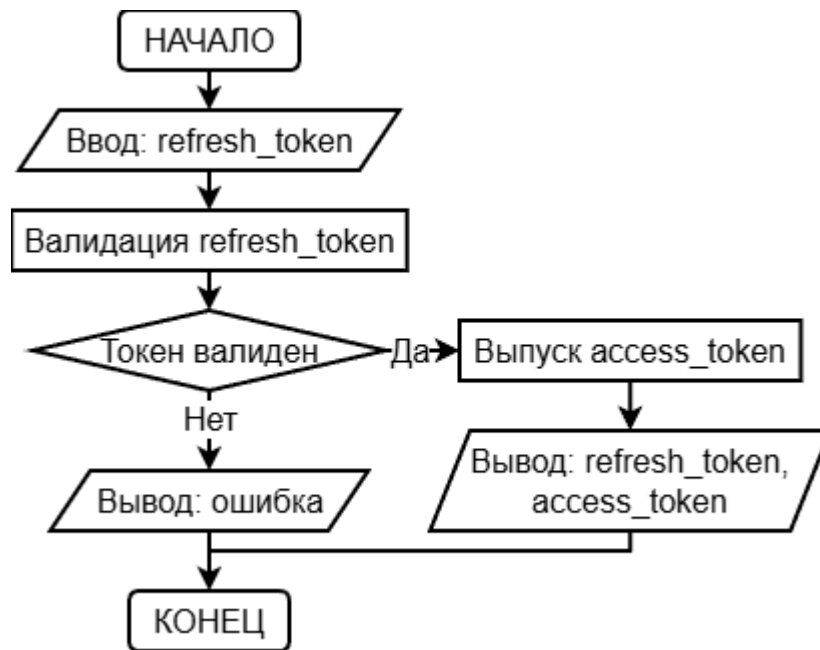


Рисунок 3.11 – блок-схема алгоритма обновления access-токена

Когда подходит срок годности refresh-токена пользователю для продолжения работы в системе необходимо пройти двухэтапную процедуру входа. На рисунках 3.12 и 3.13 – блок-схемы соответствующих алгоритмов.

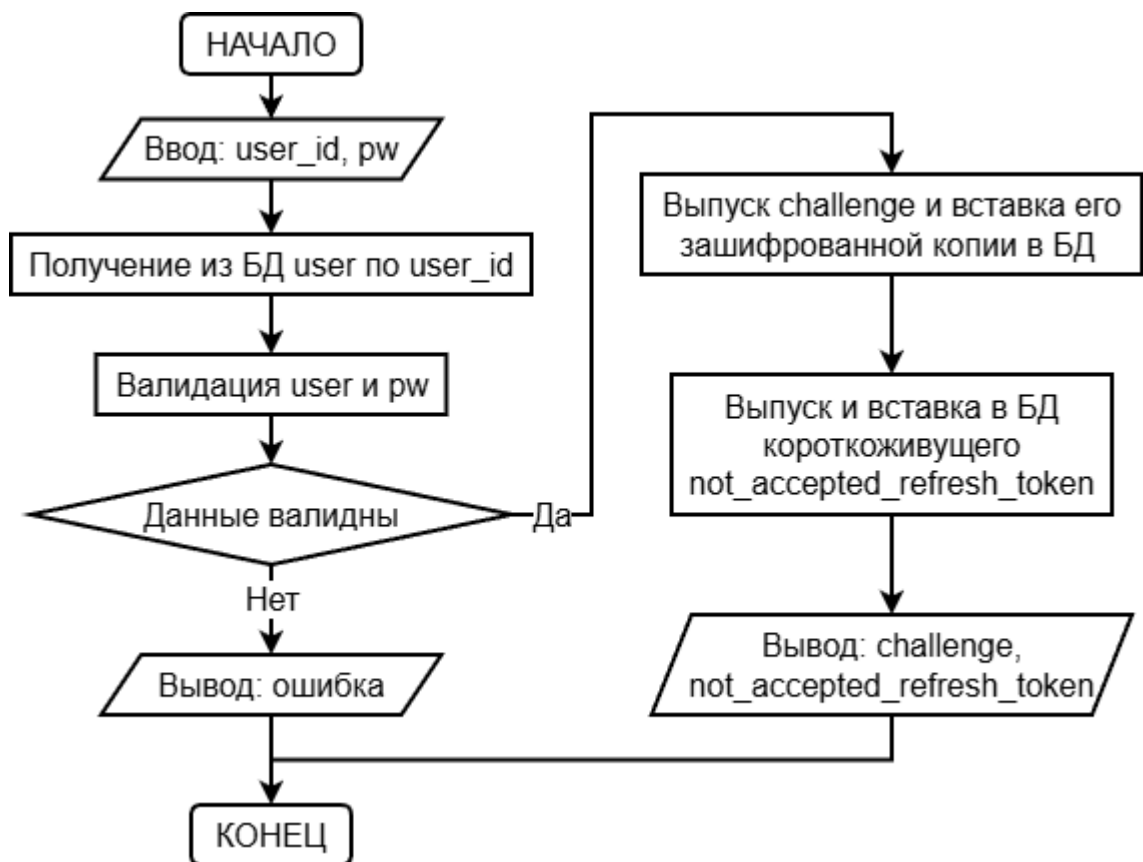


Рисунок 3.12 – блок-схема алгоритма инициализации входа пользователя

Как видно из рисунка, в рамках инициализации пользователь предоставляет свой ID и пароль, данные проходят верификацию. В случае успеха и если пользователь не заблокирован, выпускается challenge на основе случайной строки и временной метки, после чего он зашифровывается на секретном ключе сервера и сохраняется в БД. Далее выпускается и вставляется в БД неподтвержденный короткоживущий refresh-токен. Копия challenge зашифровывается на ключе пользователя и вместе с выпущенным refresh-токеном выводится пользователю.

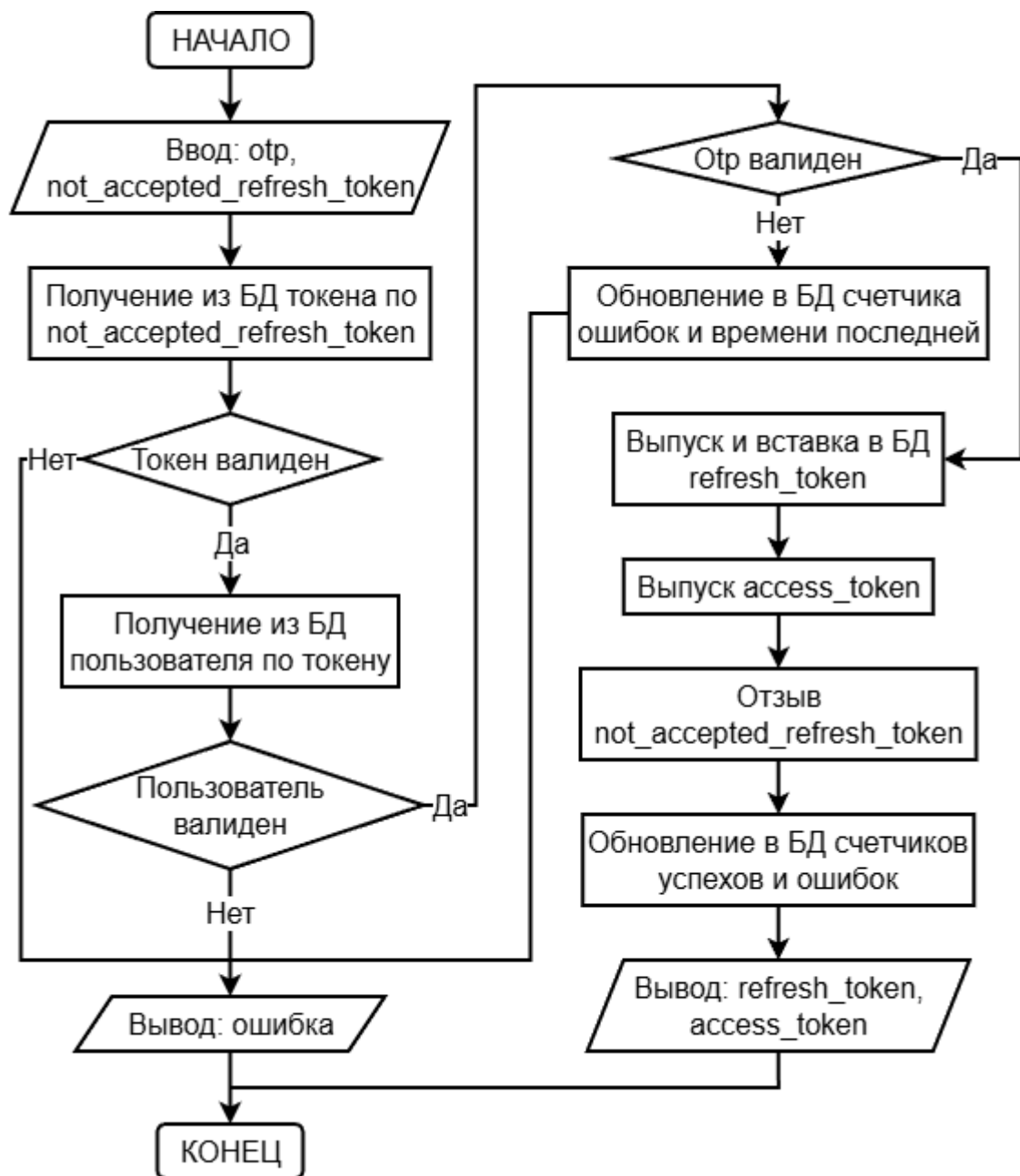


Рисунок 3.13 – блок-схема алгоритма подтверждения входа пользователя

Алгоритм подтверждения регистрации начинается с того, что пользователь вводит вычисленный OTP и полученный ранее refresh-токен, который сразу же проверяется на соответствие требованиям: короткоживущий, неподтвержденный, неотозванный и непросроченный. В случае успеха из БД получается связанная учетная запись пользователя, которая должна быть подтвержденной и незаблокированной. Если все корректно, производится расшифрование OTP. В случае успеха извлекаются составные части: случайная строка (которая ранее добавлялась в challenge) и счетчик успехов. Части проверяются на соответствие эталонным значениям. Если проверка завершилась успехом, то далее алгоритм продолжается обновлением счетчиков (успеха и неудач), отзывом старого refresh-токена, выпуском новой пары (refresh и access) и их отправкой пользователю. Если же проверка OTP завершилась неудачей, вызывается специальная функция-обработчик, фиксирующая это событие (при нескольких подряд неудачах пользователь временно блокируется).

Последняя из функций сервера – выход пользователя (рисунок 3.14).

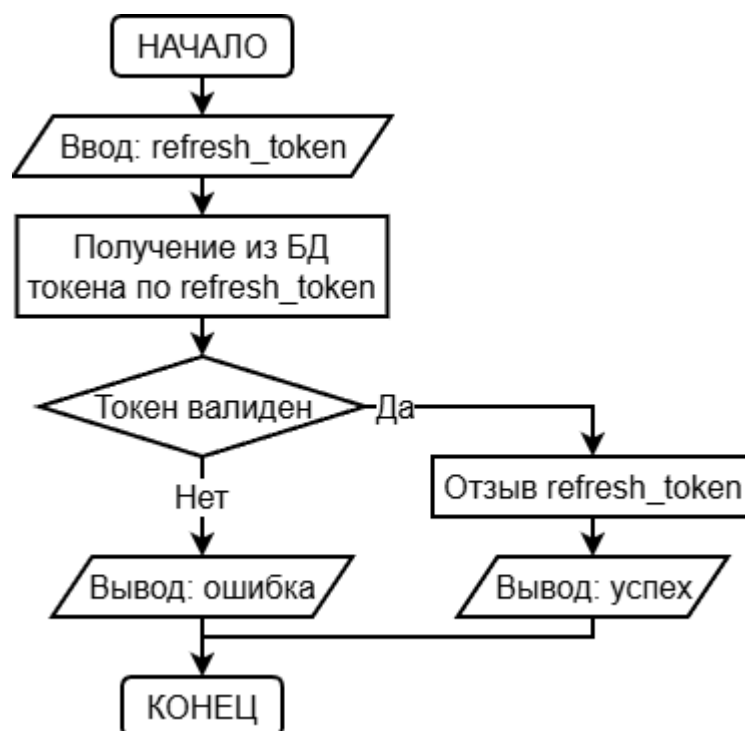


Рисунок 3.14 – блок-схема алгоритма выхода пользователя

Все, что она делает: верифицирует полученный от пользователя refresh-токен. Если токен подтвержден, а связанная учетная запись пользователя активирована, токен отзывается. Сообщение о статусе направляется пользователю.

### 3.4 Технологический стек

При разработке системы аутентификации критически важно выбрать оптимальный технологический стек, обеспечивающий безопасность, производительность и масштабируемость (таблица 3.1).

Таблица 3.1 – Технологический стек проектируемой MFA-системы

| Компонент  | Назначение         | Технология | Краткое обоснование                                                                                                                 |
|------------|--------------------|------------|-------------------------------------------------------------------------------------------------------------------------------------|
| БД         | СУБД               | PostgreSQL | Надежность, поддержка ACID, масштабируемость, свободная лицензия                                                                    |
| ОТР-токен  | ЯП                 | Kotlin     | Де-факто стандарт современной Android-разработки                                                                                    |
| Сервер     | ЯП                 | Python     | Простота разработки и поддержки, асинхронность, развитая экосистема                                                                 |
| Сервер     | Веб-API библиотека | FastAPI    | Высокая производительность, асинхронность, легковесность, поддержка OpenAPI и JWT                                                   |
| Сервер     | ORM библиотека     | SQLAlchemy | Гибкость, поддержка PostgreSQL, развитая экосистема, высокая производительность, возможность исполнения низкоуровневых SQL-запросов |
| БД, сервер | Контейнеризация    | Docker     | Простота развертывания, масштабируемость, развитая экосистема                                                                       |

### **3.5 Итоги**

В текущей главе производилось проектирование системы мультифакторной аутентификации. Сперва были предъявлены требования к проектируемой MFA-системе. Далее, согласно требованиям, были определены необходимые компоненты: ОТР-токен, база данных и сервер аутентификации. Была спроектирована схема БД и построена диаграмма ее сущностей-связей. После были подробно описаны алгоритмы работы компонентов системы, отраженные на соответствующих блок-схемах. Наконец, был представлен технологический стек разработки системы с кратким обоснованием выбора конкретных технологий: СУБД, ЯП, библиотеки и система контейнеризации.

Таким образом, можно констатировать, что MFA-система полностью спроектирована, а значит, очередная задача успешно выполнена.

## 4 Программная реализация спроектированной системы

Следом за проектированием начинается этап программной реализации. Данная глава посвящена написанию программного кода и конфигурированию системы разворачивания основных компонентов разрабатываемой системы мультимакторной аутентификации.

### 4.1 БД аутентификации

Под программной реализацией базы данных обычно понимается написание sql-скриптов, создающих необходимые таблицы. Учитывая, что в текущем проекте используется ORM-библиотека на сервере аутентификации, создание таблиц производится ручным вызовом скрипта `init_db.py` после первого запуска сервисов. В данном же подразделе описано разворачивание сервера с СУБД внутри Docker-контейнера.

На рисунке 4.1 представлен скриншот из IDE PyCharm, на котором видно фрагмент файлового проводника. В корневой директории – два файла: `.env` (для переменных окружения) и `docker-compose.yml` (для разворачивания многоконтейнерных Docker-проектов). В той же директории также присутствует `.secrets/` (для файлов с секретами: паролями, секретными ключами, сертификатами).

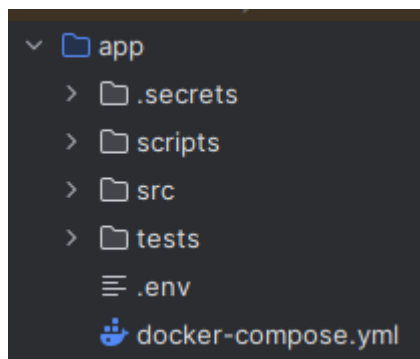


Рисунок 4.1 – фрагмент файловой структуры проекта

Рассмотрим фрагмент файла `docker-compose.yml` (рисунок 4.2). В нем присутствует описание сервиса `db` на основе официального образа PostgreSQL

17. Из примечательного – пароль от БД передается через механизм docker secrets, чтобы избежать логирования секрета. Значения для переменных окружения контейнера берутся значения из .env.

```
docker-compose.yml x
3   services:
4     db:
5       image: postgres:17
6       container_name: auth_db
7       restart: unless-stopped
8       environment:
9         POSTGRES_USER: ${DB_USER}
10        POSTGRES_DB: ${DB_NAME}
11        POSTGRES_PASSWORD_FILE: /run/secrets/db_password
12       ports:
13         - "${DB_PORT}:${DB_PORT}" # todo УДАЛИТЬ на проде
14       secrets:
15         - db_password
16       volumes:
17         - postgres_data:/var/lib/postgresql/data
18       networks:
19         - auth_net
20       healthcheck:
21         test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
22         interval: 5s
23         timeout: 3s
24         retries: 5
25
26 > server: <9 keys>
49
50     networks:
51       auth_net:
52         driver: bridge
53
54     volumes:
55       postgres_data:
56
57     secrets:
58       db_password:
59         file: .secrets/db_password
```

Рисунок 4.2 – фрагмент файла docker-compose.yml

На рисунке 4.3 представлен фрагмент файла .env, в котором указываются конкретные значения для переменных окружения. Их централизованное хранение значительно упрощает поддержку проекта, позволяя быстро изменять параметры компонентов системы.

```
9      # DB
10     DB_NAME=auth_db
11     DB_HOST=db
12     DB_PORT=5432
13     DB_USER=auth_user
```

Рисунок 4.3 – фрагмент файла .env

## 4.2 Сервер аутентификации

С учетом выбранного технологического стека программная реализация сервера аутентификации происходит в два этапа: конфигурирование системы автоматизированного развертывания и непосредственно написание программного кода.

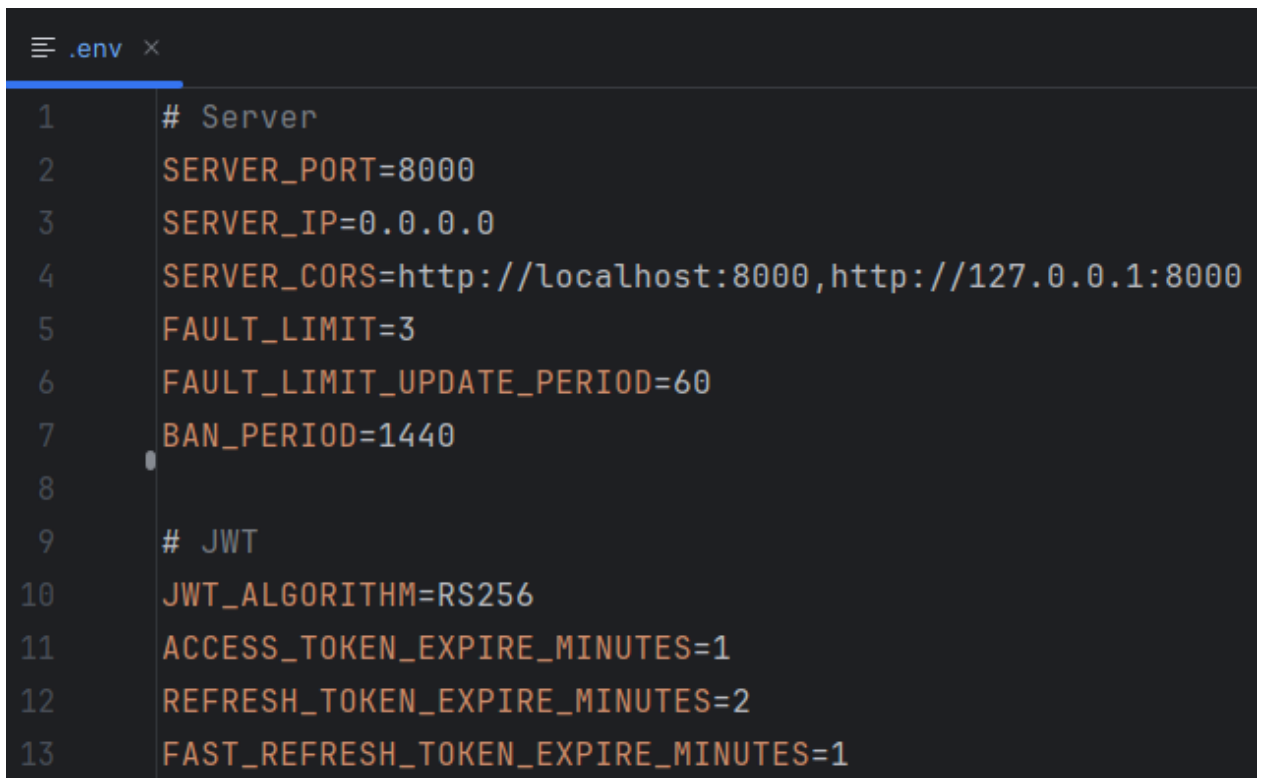
### 4.2.1 Развертывание

Как и в случае БД для сервера аутентификации в качестве системы контейнеризации используется docker. На рисунке 4.4 – фрагмент файла docker-compose.yml, описывающий конфигурирование и запуск сервера аутентификации.

Из рисунка видно, что секретные данные (сертификаты для подписи JWT, мастер-ключ сервера и пароль для доступа к БД), как и для db, передаются через docker secrets. Остальные же данные передаются на сервер через переменные окружения из .env (рисунок 4.5).

```
docker-compose.yml x
3   services:
25
26   server:
27     build: .
28     container_name: auth_server
29     restart: unless-stopped
30     env_file:
31       - .env
32     environment:
33       DB_PASSWORD_FILE: /run/secrets/db_password
34       SECRET_KEY_FILE: /run/secrets/secret_key
35       JWT_PUBLIC_KEY_FILE: /run/secrets/jwt_public_key
36       JWT_PRIVATE_KEY_FILE: /run/secrets/jwt_private_key
37     secrets:
38       - db_password
39       - secret_key
40       - jwt_public_key
41       - jwt_private_key
42     ports:
43       - "${SERVER_PORT}:${SERVER_PORT}"
44     depends_on:
45       db:
46         condition: service_healthy
47     networks:
48       - auth_net
49   > networks: auth_net
52   > volumes: postgres_data
54   secrets:
55   > db_password: <1 key>
57   secret_key:
58     file: .secrets/secret_key
59   jwt_public_key:
60     file: .secrets/jwt_public_key.pem
61   jwt_private_key:
62     file: .secrets/jwt_private_key.pem
```

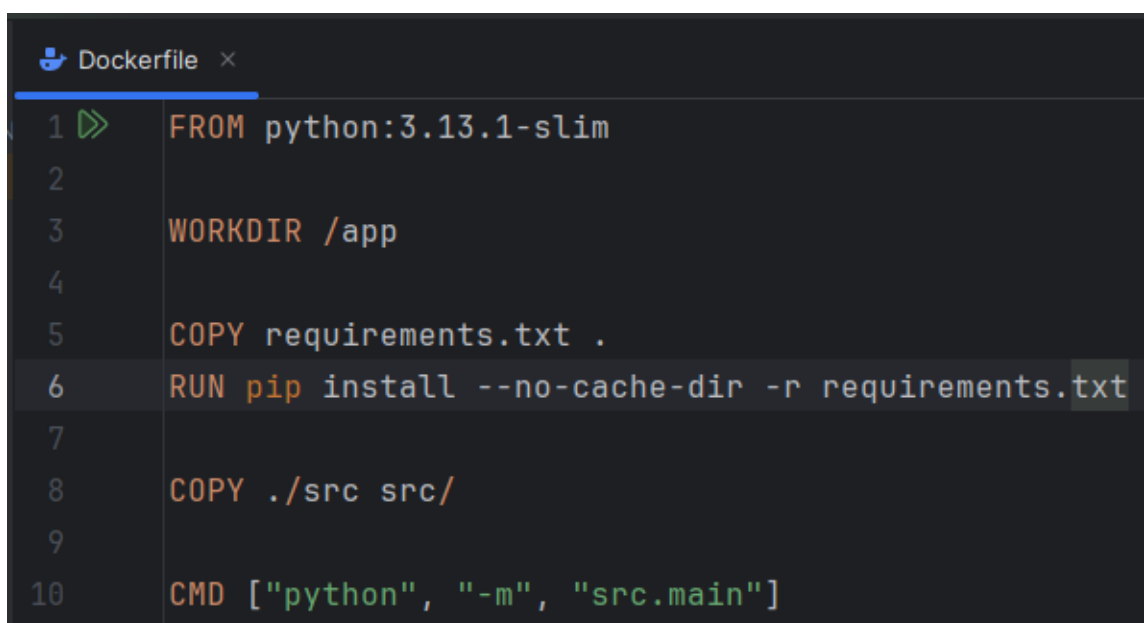
Рисунок 4.4 – фрагмент файла docker-compose.yml



```
1 # Server
2 SERVER_PORT=8000
3 SERVER_IP=0.0.0.0
4 SERVER_CORS=http://localhost:8000,http://127.0.0.1:8000
5 FAULT_LIMIT=3
6 FAULT_LIMIT_UPDATE_PERIOD=60
7 BAN_PERIOD=1440
8
9 # JWT
10 JWT_ALGORITHM=RS256
11 ACCESS_TOKEN_EXPIRE_MINUTES=1
12 REFRESH_TOKEN_EXPIRE_MINUTES=2
13 FAST_REFRESH_TOKEN_EXPIRE_MINUTES=1
```

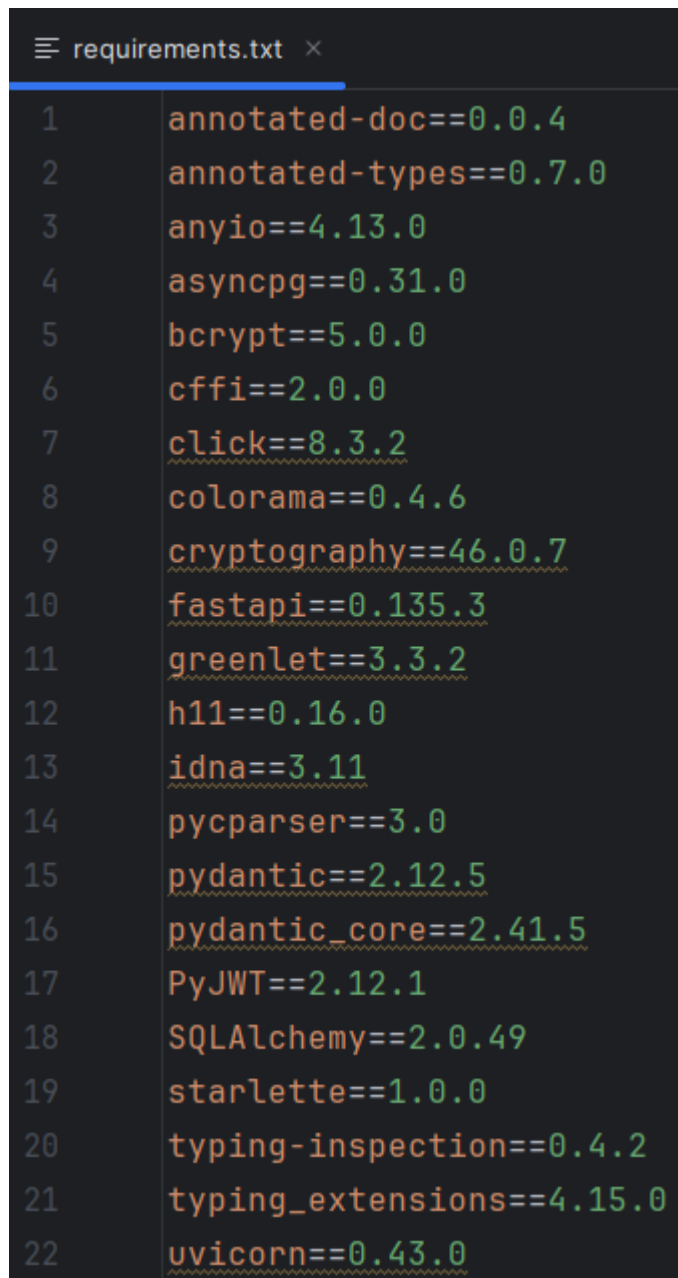
Рисунок 4.5 – фрагмент файла .env

Как видно из содержимого файла docker-compose.yml, сборка образа контейнера для сервера происходит в той же директории. На рисунке 4.6 представлен Dockerfile, в котором определяется базовый образ контейнера, рабочая директория, установка зависимостей для сервера из файла requirements.txt (рисунок 4.7) и копирование файлов с исходным кодом.



```
1 FROM python:3.13.1-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install --no-cache-dir -r requirements.txt
7
8 COPY ./src src/
9
10 CMD ["python", "-m", "src.main"]
```

Рисунок 4.6 – файл Dockerfile

A screenshot of a code editor window titled 'requirements.txt'. The editor displays a list of 22 Python dependencies, each on a new line. The dependencies are: annotated-doc==0.0.4, annotated-types==0.7.0, anyio==4.13.0, asynccpg==0.31.0, bcrypt==5.0.0, cffi==2.0.0, click==8.3.2, colorama==0.4.6, cryptography==46.0.7, fastapi==0.135.3, greenlet==3.3.2, h11==0.16.0, idna==3.11, pycparser==3.0, pydantic==2.12.5, pydantic\_core==2.41.5, PyJWT==2.12.1, SQLAlchemy==2.0.49, starlette==1.0.0, typing-inspection==0.4.2, typing\_extensions==4.15.0, and uvicorn==0.43.0. The text is color-coded: package names are in orange, version numbers in green, and the '==' operator in light blue. Lines 7, 9, 10, 11, and 22 are underlined with a wavy line.

```
1 annotated-doc==0.0.4
2 annotated-types==0.7.0
3 anyio==4.13.0
4 asynccpg==0.31.0
5 bcrypt==5.0.0
6 cffi==2.0.0
7 click==8.3.2
8 colorama==0.4.6
9 cryptography==46.0.7
10 fastapi==0.135.3
11 greenlet==3.3.2
12 h11==0.16.0
13 idna==3.11
14 pycparser==3.0
15 pydantic==2.12.5
16 pydantic_core==2.41.5
17 PyJWT==2.12.1
18 SQLAlchemy==2.0.49
19 starlette==1.0.0
20 typing-inspection==0.4.2
21 typing_extensions==4.15.0
22 uvicorn==0.43.0
```

Рисунок 4.7 – файл requirements.txt

#### 4.2.2 Компоненты и классы

Для упрощения дальнейшей поддержки сервера было принято решение придерживаться слоистой архитектуры при его разработке. На рисунке 4.8 представлена UML-диаграмма компонентов, из которых относящиеся непосредственно к серверу являются его основными слоями.

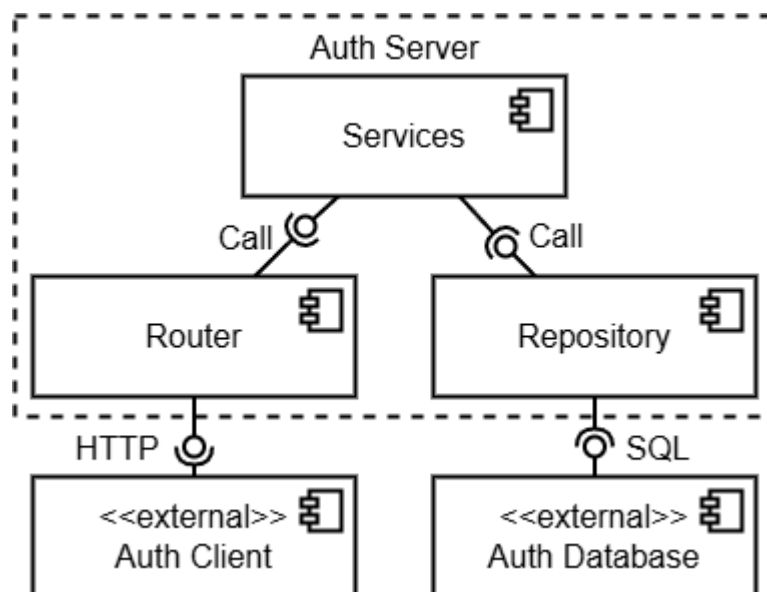


Рисунок 4.8 – UML-диаграмма компонентов-слоев сервера

Как видно из рисунка, слой Router предоставляет интерфейс HTTP-запросов для внешнего клиента аутентификации (внешней инфраструктуры). Задачей данного слоя является вызов определенных функций сервисного слоя в зависимости от конкретных входящих HTTP-запросов.

За предоставления основной бизнес-логики сервера отвечает компонент Services, который в свою очередь состоит из статических классов, представленных на рисунке 4.9:

- SystemService – предоставляет системные услуги (healthcheck);
- RegisterService – регистрация пользователя;
- LoginService – вход пользователя;
- UserService – валидация и проверка ограничений;
- TokenService – валидация, выпуск и отзыв refresh-токенов и обновление access-токена.

Как видно из рисунка, классы сервисного слоя зависят от двух вспомогательных слоев, которые не были отражены на диаграмме компонентов сервера в виду сравнительно малой значимости.

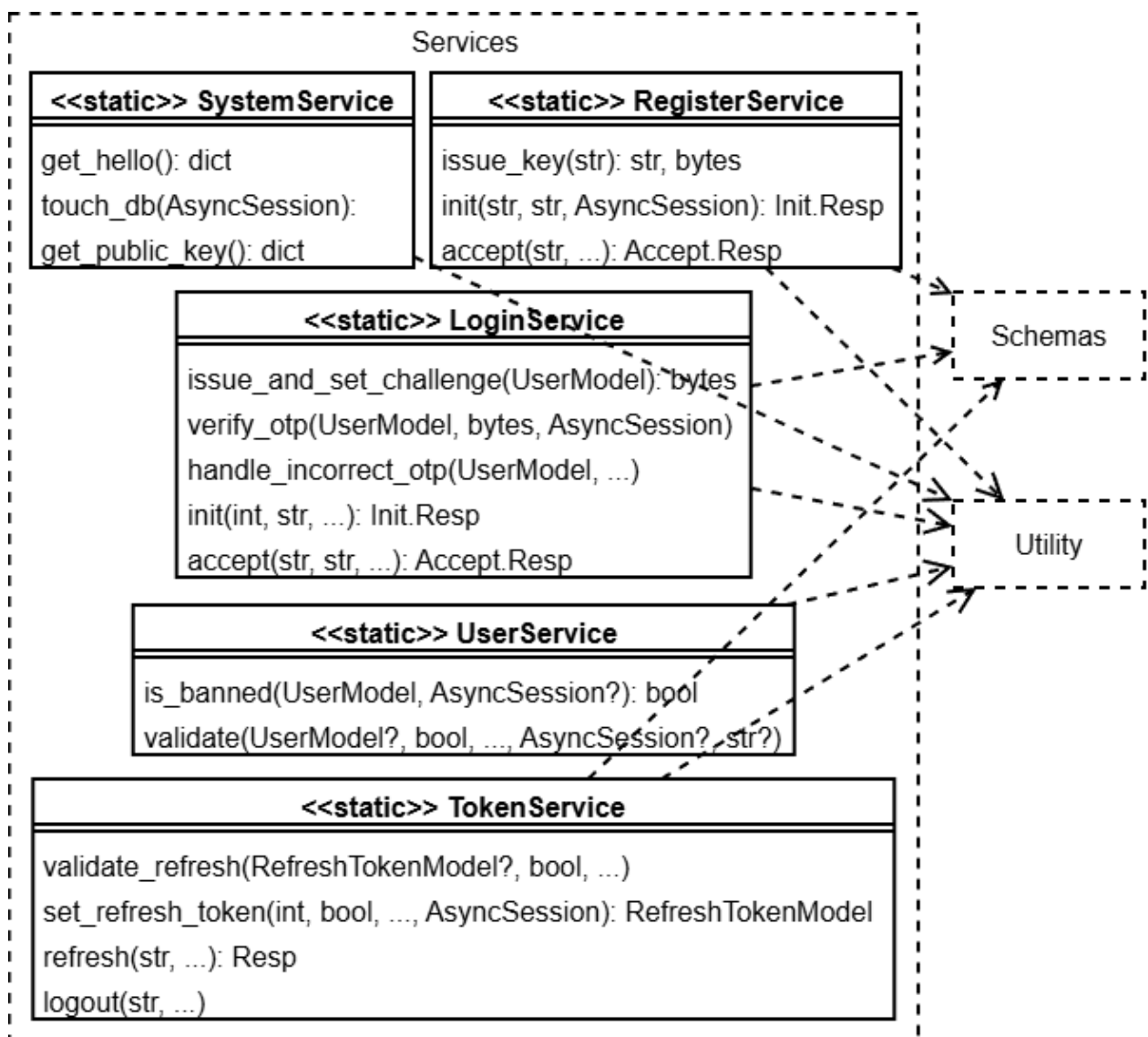


Рисунок 4.9 – UML-диаграмма классов сервисного слоя

Слой Schemas представляет собой набор Pydantic-классов, используемых для валидации передаваемых данных (рисунок 4.10). Utility же инкапсулирует синглтон Settings (параметры сервера) и статические утилитарные классы (рисунок 4.11):

- JWT – работа с токенами;
- SymmetricEncryption – симметричное шифрование;
- Random – ГСЧ;
- Password – хэширование и верификация паролей;
- Utils – прочие утилиты.

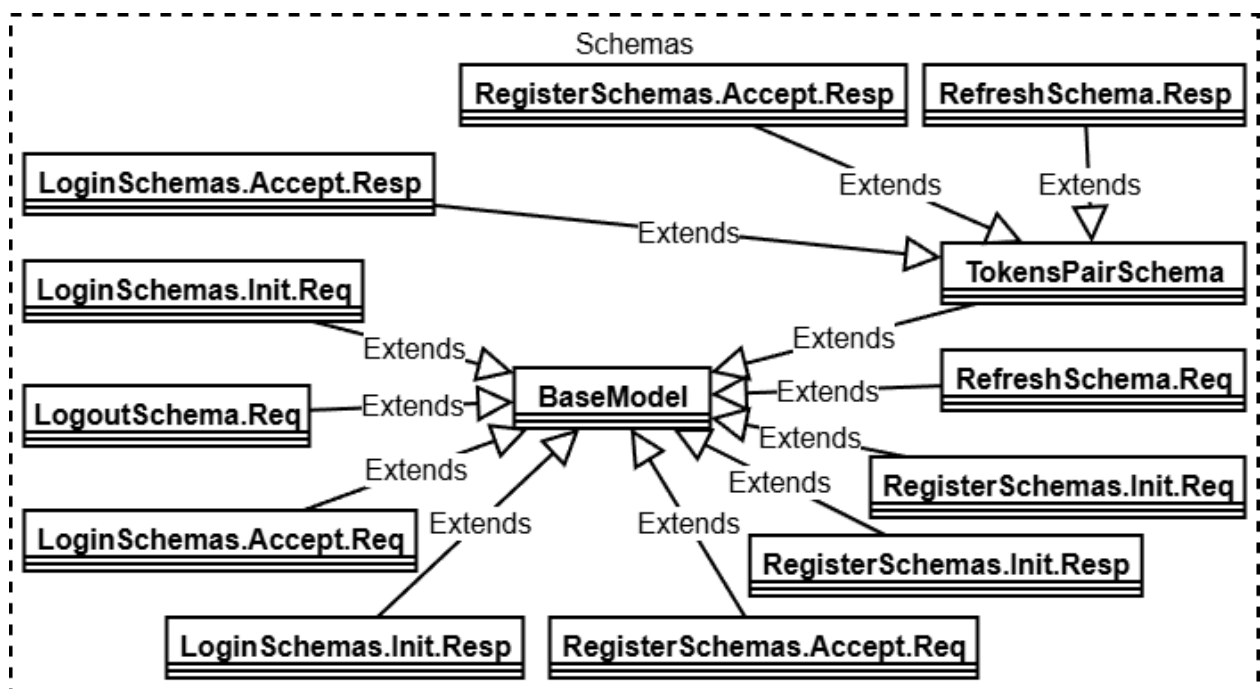


Рисунок 4.10 – UML-диаграмма классов слоя Pydantic-схем

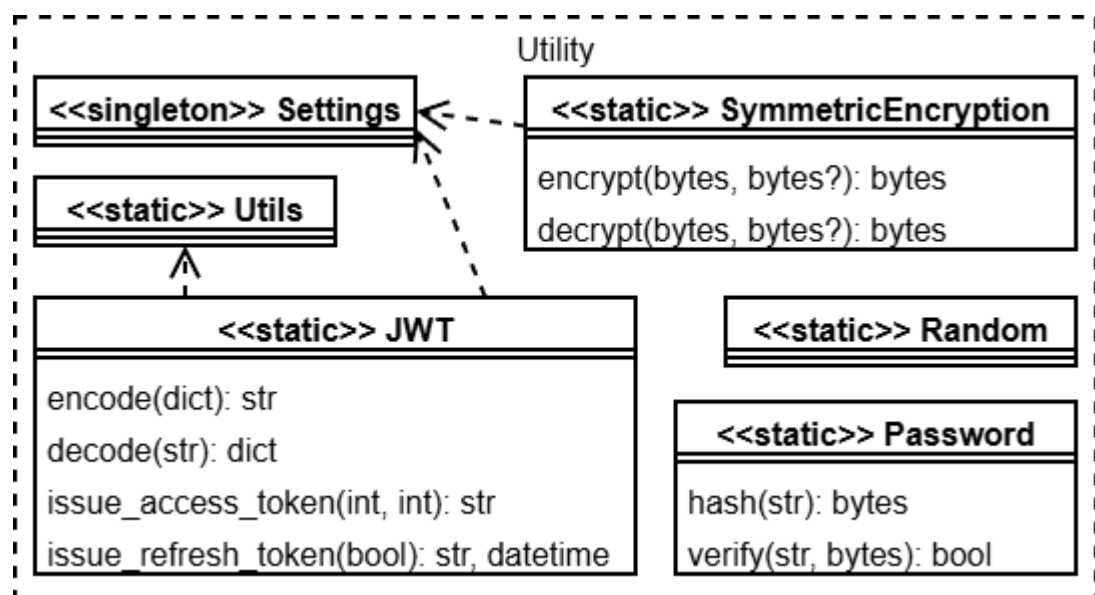


Рисунок 4.11 – UML-диаграмма классов слоя утилит

Наконец, последний (но не по значимости) слой – Repository выступает прослойкой между сервисным слоем (предоставляя ему интерфейс вызова функций) и внешним с точки зрения сервера компонентом базы данных (требуя от него интерфейс SQL-запросов). На рисунке 4.12 – UML-диаграмма классов этого слоя, включающая:

- DeclarativeBase – базовый класс объектов таблиц в SQLAlchemy;

- Base – собственный базовый класс для таблиц, наследуется от DeclarativeBase, используется для создания сущностей и их связей в БД;
- UserModel – объекты записей таблицы users;
- RefreshTokenModel – объекты записей таблицы refresh\_tokens;
- AsyncAuthRepo – статический, асинхронные функции CRUD-операций.

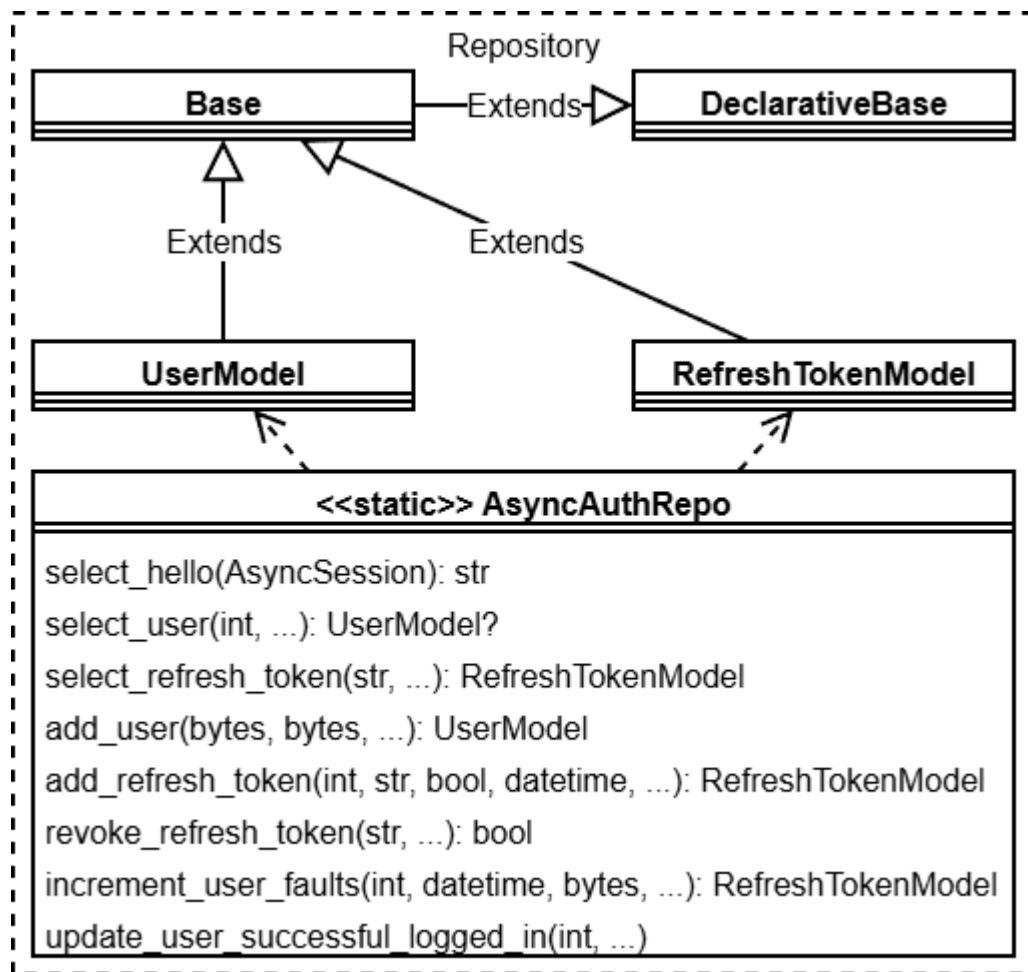


Рисунок 4.12 – UML-диаграмма классов слоя Repository

В данном разделе не приводится диаграмма классов Router-слоя, так как он представляет из себя простой набор асинхронных функций, реализующих REST-API и для каждого из предусмотренных эндпоинтов вызывающих соответствующую функцию сервисного слоя.

### 4.3 ОТР-токен

Как уже упоминалось ранее, ОТР-токен представляет собой Android-приложение, написанное на Kotlin, наследовавший от Java обязательность применения ООП при написании программ.

На рисунке 4.13 представлена UML-диаграмма классов, обеспечивающих регистрацию токена. Среди них:

- два фрагмента, предназначенных для инициализации и подтверждения регистрации;
- RegistrationViewModel, хранящий состояние токена в процессе регистрации.

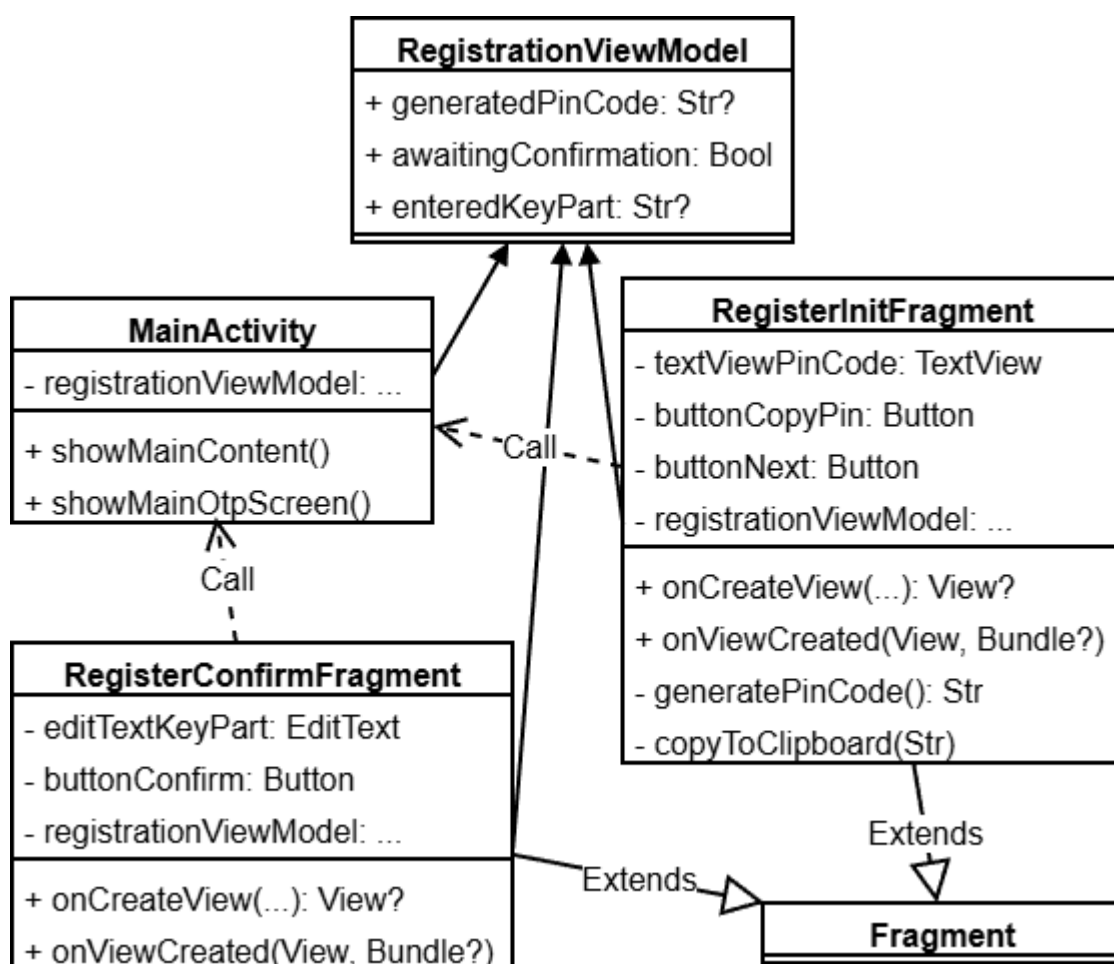


Рисунок 4.13 – UML-диаграмма классов регистрации ОТР-токена

На рисунке 4.14 – классы, обеспечивающие функционал токена при его активации. Методы MainOtpFragment реализуют основную логику процесса.

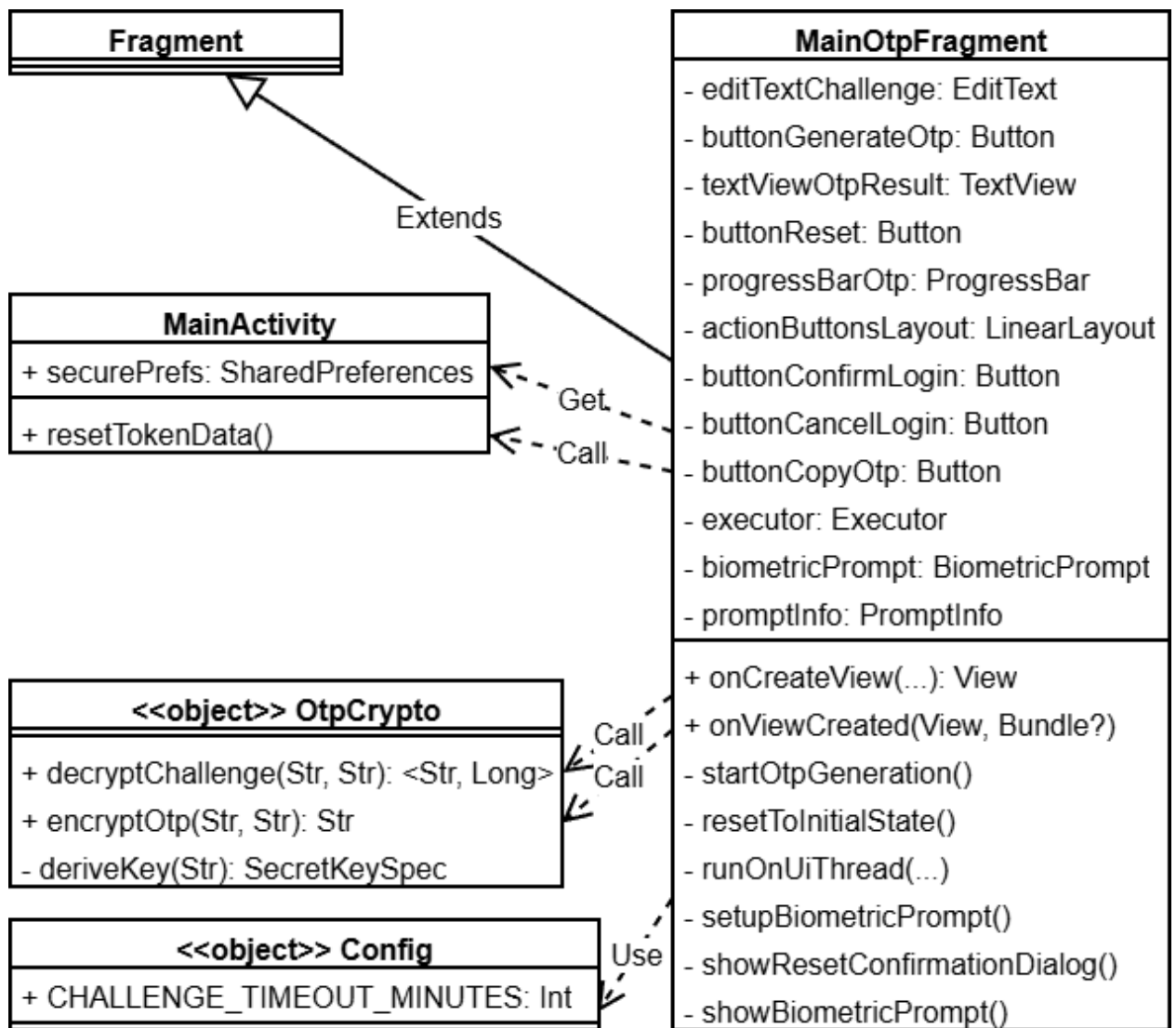


Рисунок 4.14 – UML-диаграмма классов активации OTP-токена

На рисунке 4.15 – классы, связанные с основной activity OTP-токена. Согласно спецификации, класс MainActivity является точкой входа для любого Android-приложения. В случае разрабатываемого токена из данного класса вызывается диалоговое окно для биометрической аутентификации, после чего в зависимости от состояния токена открывается окно либо регистрации, либо активации OTP-токена.

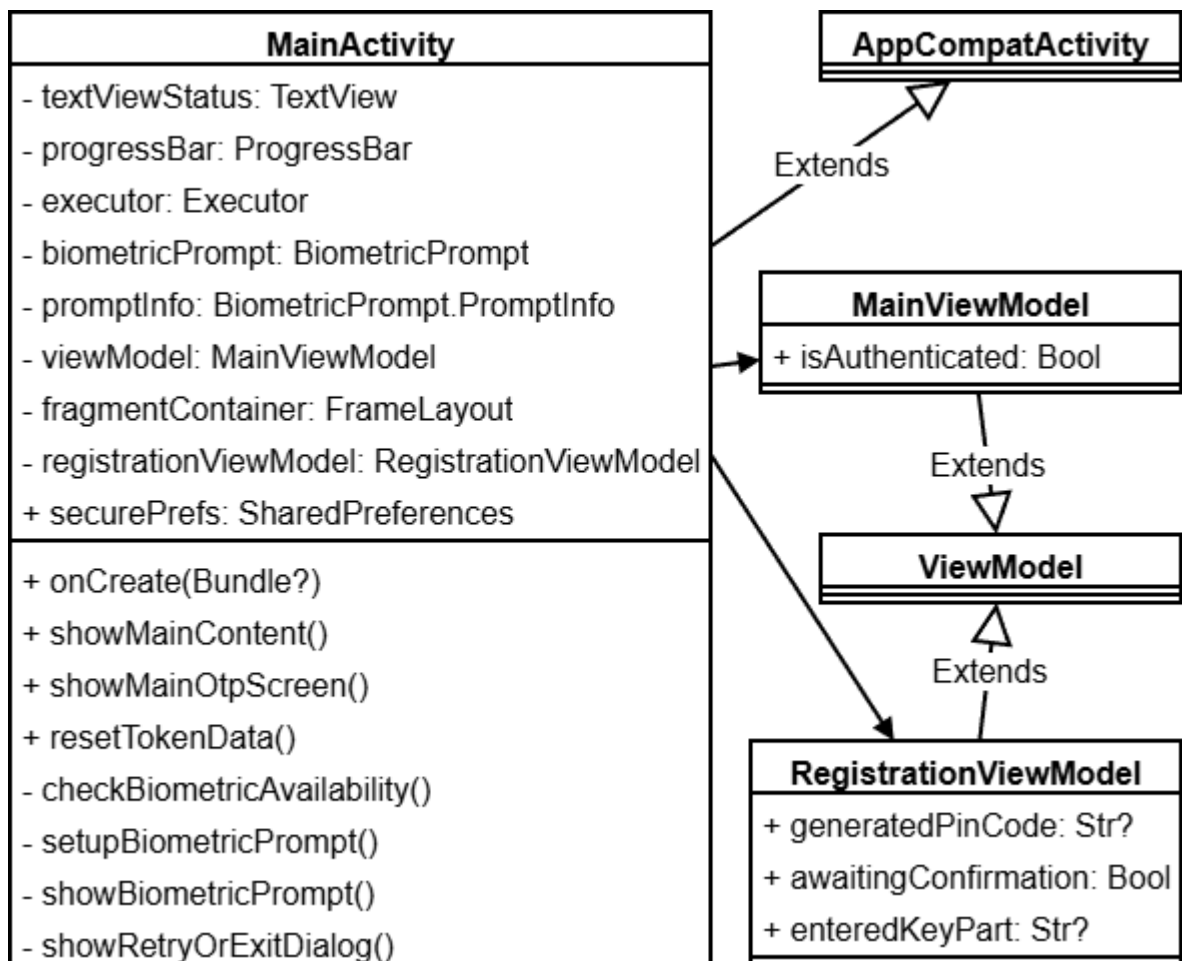


Рисунок 4.15 – UML-диаграмма классов, связанных с точкой входа в приложение

#### 4.4 Итоги

В данной главе была подробно описана конфигурация для развертывания основных компонентов разрабатываемой системы мультифакторной аутентификации с помощью Docker. Компоненты разворачиваются в виде сервисов, объединенных в общую сеть. Для передачи в контейнеры нечувствительных данных используются переменные окружения, в то время как для секретных – docker secrets.

Далее, был написан python-модуль, содержащий файлы с исходным кодом для сервера аутентификации. При их написании было решено придерживаться слоистой архитектуры, применяя принципы ООП. Например, для взаимодействия с базой данных был реализован слой Repository,

инкапсулирующий модели, созданные на основе ORM-библиотеки SQLAlchemy.

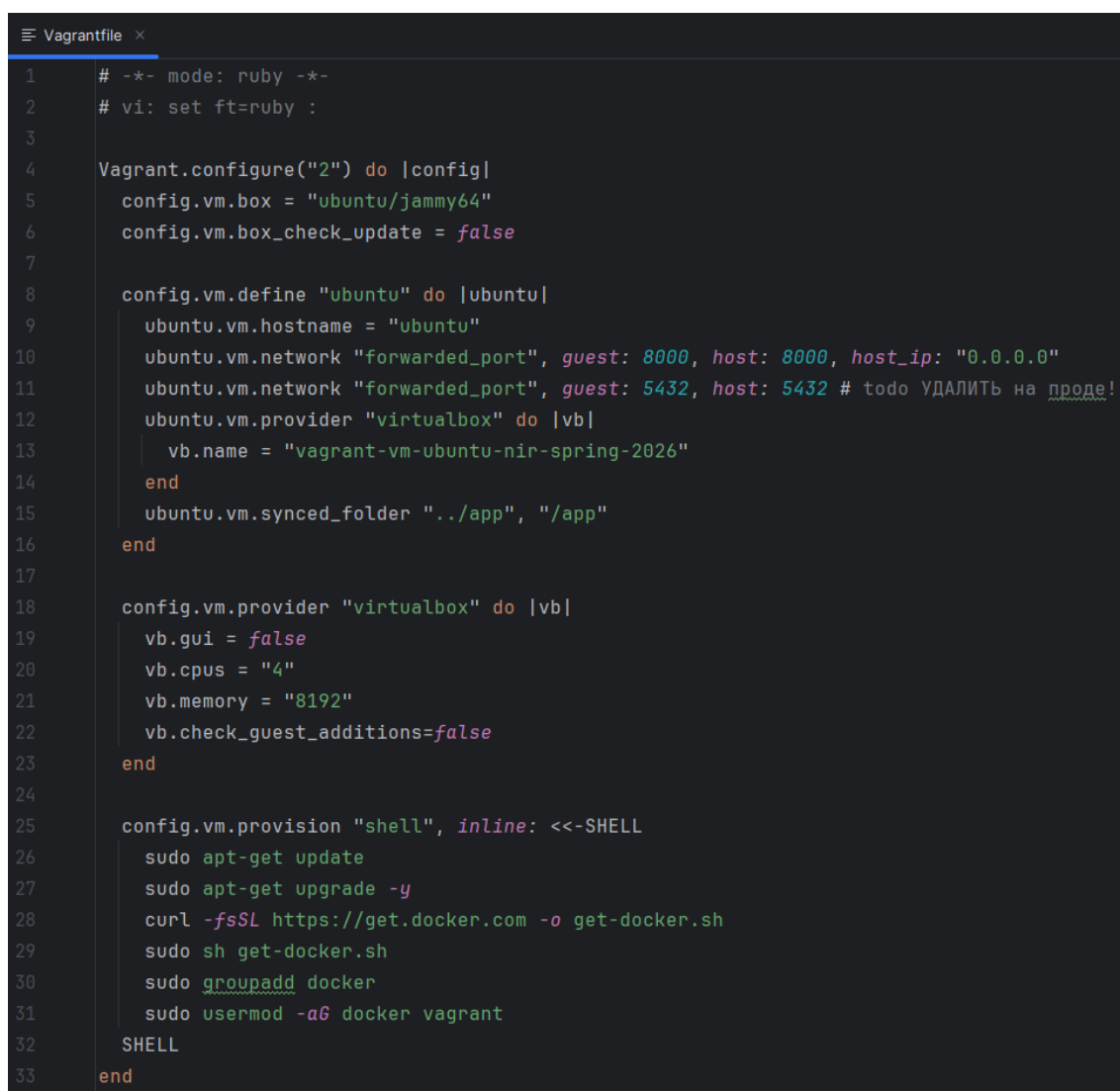
Наконец, на языке Kotlin также согласно принципам ООП было реализовано Android-приложение, выполняющего роль OTP-токена в разрабатываемой системе multifactorной аутентификации.

## 5 Тестирование и рефакторинг реализованной системы

В цикле создания сложных программных решений обязательно должен присутствовать этап тестирования и рефакторинга разрабатываемых продуктов. В данной главе производится интеграционное, нагрузочное и комплексное тестирование реализованной системы.

### 5.1 Виртуальная лаборатория

В качестве экспериментальной лаборатории использовалась виртуальная машина, которая конфигурировалась и запускалась при помощи инструмента Vagrant (рисунок 5.1).



```
1  # -*- mode: ruby -*-
2  # vi: set ft=ruby :
3
4  Vagrant.configure("2") do |config|
5    config.vm.box = "ubuntu/jammy64"
6    config.vm.box_check_update = false
7
8    config.vm.define "ubuntu" do |ubuntu|
9      ubuntu.vm.hostname = "ubuntu"
10     ubuntu.vm.network "forwarded_port", guest: 8000, host: 8000, host_ip: "0.0.0.0"
11     ubuntu.vm.network "forwarded_port", guest: 5432, host: 5432 # todo УДАЛИТЬ на проде!
12     ubuntu.vm.provider "virtualbox" do |vb|
13       vb.name = "vagrant-vm-ubuntu-nir-spring-2026"
14     end
15     ubuntu.vm.synced_folder "../app", "/app"
16   end
17
18   config.vm.provider "virtualbox" do |vb|
19     vb.gui = false
20     vb.cpus = "4"
21     vb.memory = "8192"
22     vb.check_guest_additions=false
23   end
24
25   config.vm.provision "shell", inline: <<-SHELL
26     sudo apt-get update
27     sudo apt-get upgrade -y
28     curl -fsSL https://get.docker.com -o get-docker.sh
29     sudo sh get-docker.sh
30     sudo groupadd docker
31     sudo usermod -aG docker vagrant
32   SHELL
33 end
```

Рисунок 5.1 – конфигурационный файл Vagrantfile

Как видно из конфигурационного файла, ВМ создавалась на основе 64-битного образа дистрибутива Ubuntu Jammy. Для доступа с хостовой системы к API сервера и БД были проброшены необходимые порты. Для работы сервисов ВМ было выделено 4 CPU и 8 GiB RAM. Также для удобства была прокинута общая директория с исходниками сервера и БД. Для первичного обновления системы и установки docker был написан провижн-скрипт, выполняемый при создании ВМ или при запуске со специальным флагом.

## 5.2 Интеграционное тестирование

Интеграционное тестирование API позволяет проверить корректность взаимодействия сервера аутентификации и базы данных при обработке HTTP-запросов. В отличие от модульных тестов, проверяющих изолированные функции, интеграционные тесты выполняют реальные запросы к работающему серверу и фиксируют время ответа, что даёт объективную картину производительности системы.

Для автоматизации тестирования использовался фреймворк pytest в связке с синхронным HTTP-клиентом httpx. Выбор синхронного клиента обусловлен простотой написания линейных сценариев и отсутствием необходимости в параллельном выполнении запросов внутри одного теста. Все тесты объединены в файл test\_api.py, вспомогательные функции и фикстуры вынесены в conftest.py.

Инфраструктура тестов включает несколько ключевых фикстур. Фикстура client создаёт один HTTP-клиент на всю тестовую сессию, что позволяет использовать keep-alive соединение и исключить накладные расходы на повторную установку TCP-соединения. Перед началом содержательных тестов выполняется «прогревочный» запрос к эндпоинту /auth/touch-db, который инициализирует пул соединений с базой данных и «разогревает» сервер. Фикстура registered\_user инкапсулирует полный цикл регистрации

нового пользователя (init + accept) и возвращает словарь с идентификатором пользователя, паролем, refresh- и access-токенами, а также временами выполнения каждого этапа регистрации. Для измерения времени используется атрибут `resp.elapsed.total_seconds()`, значение которого умножается на 1000 для получения миллисекунд.

Тестовые сценарии покрывают все основные эндпоинты сервера аутентификации. Для регистрации проверяется как успешная инициализация (POST /auth/register-init с валидными `pin_code` и `password`), так и успешное подтверждение (POST /auth/register-accept с корректным `refresh_token`), а также негативный сценарий с недействительным токеном. Инициализация входа тестируется с корректными учётными данными, с неверным паролем и с несуществующим идентификатором пользователя. Подтверждение входа с корректным OTP сознательно не автоматизировано, поскольку для его вычисления требуется воспроизвести криптографические операции клиента (расшифрование `challenge`, формирование OTP, шифрование результата), что дублирует логику Android-приложения и выходит за рамки тестирования серверных компонентов. Вместо этого проверяется, что сервер корректно отвергает заведомо некорректный OTP, возвращая статус 401, а также обрабатывает попытку подтверждения без предварительной инициализации.

Дополнительно тестируются операции обновления (POST /auth/refresh) и валидации access-токена (GET /auth/validate-access-token), выхода (DELETE /auth/logout), а также сценарий с отозванным refresh-токеном. Отдельный тест проверяет получение публичного ключа (GET /auth/public-key) и верификацию подписи access-токена, полученного при регистрации.

Все результаты измерений автоматически сохраняются в CSV-файл.

В ходе первого прогона интеграционных тестов была выявлена ошибка в серверной логике: при передаче заведомо некорректного OTP (строка, не являющаяся валидным Base64 или имеющая недостаточную длину после

декодирования) метод `SymmetricEncryption.decrypt` выбрасывал необработанное исключение `ValueError`, что приводило к ответу с кодом 500, что неверно отражает суть ошибки (некорректный OTP – ошибка клиента, а не сервера). Было принято решение обернуть вызов расшифрования в блок `try-except` и в случае ошибки вызывать обработчик некорректных OTP. Это изменение позволило унифицировать ответ сервера: теперь при любой некорректной попытке ввода OTP возвращается статус 401 с лаконичным сообщением «Incorrect OTP». При этом детализированная причина ошибки (неверный формат, несовпадение счётчика или challenge) логируется на стороне сервера с уровнем INFO, что не раскрывает внутреннюю логику потенциальному злоумышленнику, но сохраняет возможность аудита.

После внесения исправлений все тесты были перезапущены и успешно пройдены (рисунок 5.2). Метрики, собранные в ходе проведения тестирования в условиях сконфигурированной виртуальной лаборатории сведены в таблицу 5.1.

Таблица 5.1 – Результаты проведения интеграционного тестирования

| Название теста                     | Эндпоинт                    | Код ответа | Время (мс) |
|------------------------------------|-----------------------------|------------|------------|
| test_warmup                        | /auth/touch-db              | 200        | 2020.501   |
| test_touch_db                      | /auth/touch-db              | 200        | 2.679      |
| test_register_init_success         | /auth/register-init         | 201        | 216.930    |
| test_register_accept_success       | /auth/register-accept       | 201        | 37.998     |
| test_register_accept_invalid_token | /auth/register-accept       | 401        | 4.512      |
| test_login_init_success            | /auth/login-init            | 200        | 214.525    |
| test_login_init_wrong_password     | /auth/login-init            | 401        | 214.646    |
| test_login_init_nonexistent_user   | /auth/login-init            | 404        | 2.980      |
| test_login_accept_wrong_otp        | /auth/login-accept          | 401        | 4.615      |
| test_login_accept_without_init     | /auth/login-accept          | 401        | 3.353      |
| test_refresh_success               | /auth/refresh               | 200        | 33.391     |
| test_refresh_revoked_token         | /auth/refresh               | 401        | 2.773      |
| test_validate_access_token_valid   | /auth/validate-access-token | 200        | 1.664      |
| test_validate_access_token_invalid | /auth/validate-access-token | 401        | 3.199      |
| test_logout_success                | /auth/logout                | 204        | 3.254      |

| Название теста                             | Эндпоинт         | Код ответа | Время (мс) |
|--------------------------------------------|------------------|------------|------------|
| test_logout_revoked                        | /auth/logout     | 204        | 2.777      |
| test_public key and signature verification | /auth/public-key | 200        | 4.221      |

```

(.venv) PS C:\Inst\4_course\8_sem_2\NIR\server\app\tests> pytest test_api.py
-v
===== test session starts =====
platform win32 -- Python 3.13.1, pytest-9.0.3, pluggy-1.6.0 -- C:\Inst\4_course\8_sem_2\NIR\server\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Inst\4_course\8_sem_2\NIR\server\app\tests
plugins: anyio-4.13.0
collected 17 items

test_api.py::test_warmup PASSED [ 5%]
test_api.py::test_touch_db PASSED [ 11%]
test_api.py::test_register_init_success PASSED [ 17%]
test_api.py::test_register_accept_success PASSED [ 23%]
test_api.py::test_register_accept_invalid_token PASSED [ 29%]
test_api.py::test_login_init_success PASSED [ 35%]
test_api.py::test_login_init_wrong_password PASSED [ 41%]
test_api.py::test_login_init_nonexistent_user PASSED [ 47%]
test_api.py::test_login_accept_wrong_otp PASSED [ 52%]
test_api.py::test_login_accept_without_init PASSED [ 58%]
test_api.py::test_refresh_success PASSED [ 64%]
test_api.py::test_refresh_revoked_token PASSED [ 70%]
test_api.py::test_validate_access_token_valid PASSED [ 76%]
test_api.py::test_validate_access_token_invalid PASSED [ 82%]
test_api.py::test_logout_success PASSED [ 88%]
test_api.py::test_logout_revoked PASSED [ 94%]
test_api.py::test_public_key_and_signature_verification PASSED [100%]

===== 17 passed in 5.76s =====
(.venv) PS C:\Inst\4_course\8_sem_2\NIR\server\app\tests>

```

Рисунок 5.2 – запуск и успешное прохождение интеграционных тестов

Анализ результатов показывает, что все проверенные сценарии отработали с ожидаемыми HTTP-статусами. Время ответа для большинства запросов составляет единицы–десятки миллисекунд, что свидетельствует о высокой производительности сервера и корректной работе пула соединений с базой данных. Высокое значение (свыше 2000 мс) наблюдается только для первого «холодного» запроса. Важно отметить, что эти замеры получены в условиях двойной виртуализации (VirtualBox и Docker), которая вносит

дополнительную сетевую задержку; на реальном оборудовании можно ожидать большей производительности.

Таким образом, интеграционное тестирование подтвердило корректность реализации всех эндпоинтов сервера аутентификации, выявило и позволило оперативно устранить ошибку, а также предоставило количественные метрики времени ответа, пригодные для дальнейшего анализа.

### **5.3 Нагрузочное тестирование**

Нагрузочное тестирование предназначено для оценки поведения системы под одновременным воздействием множества пользователей и определения её пропускной способности. В отличие от интеграционных тестов, выполняющих запросы последовательно, нагрузочное тестирование имитирует параллельную работу десятков и сотен клиентов, что приближает условия эксперимента к реальной эксплуатации.

Для проведения нагрузочного тестирования был выбран фреймворк Locust, написанный на Python и использующий событийно-ориентированную модель на основе `gevent`. Это позволяет без значительных накладных расходов эмулировать сотни одновременных пользователей в рамках одного процесса. Сценарии поведения пользователей описываются в коде в виде классов, что предоставляет высокую гибкость при моделировании различных паттернов взаимодействия с сервером.

Скрипт нагрузочного тестирования (`locustfile.py`) содержит пять независимых классов виртуальных пользователей, каждый из которых реализует один типичный сценарий работы с сервером аутентификации:

- `RegisterUser` – выполняет полный цикл регистрации нового пользователя (`init` + `ассепт`) и останавливается;

- `LoginFailUser` – регистрирует пользователя, затем в цикле выполняет вход с заведомо неверным ОТР, пока не будет достигнут лимит неудачных попыток, после чего пользователь блокируется;
- `TokenValidateUser` – регистрирует пользователя и в бесконечном цикле проверяет валидность полученного access-токена, создавая постоянную нагрузку на эндпоинт верификации;
- `RefreshUser` – регистрирует пользователя и с интервалом 5–10 секунд обновляет access-токен через соответствующий эндпоинт, имитируя активного клиента с долгоживущей сессией;
- `LogoutUser` – регистрирует пользователя, выполняет выход и завершает работу.

Для устранения дублирования кода общая логика регистрации вынесена во вспомогательную функцию `register_and_get_tokens`, что упрощает поддержку скрипта. Все классы сконфигурированы с равными весами, поэтому при запуске нагрузка распределяется между сценариями равномерно (по 20% на каждый тип пользователя).

Нагрузочное тестирование проводилось на том же виртуальном стенде (VirtualBox + Docker). Параметры прогона: пик в 120 одновременных пользователей, скорость нарастания нагрузки – 2 пользователей в секунду, длительность теста – 1 минута. По окончании теста Locust сгенерировал CSV и HTML-файлы отчетов. На рисунках 5.3–5.3 представлены графики динамики основных показателей (RPS, время ответа и количество пользователей) в зависимости от времени.

Содержимое же CSV-файлов приведено в приложениях В и Г (из-за большого размера оказалось проблематично вставить его непосредственно в данном разделе).

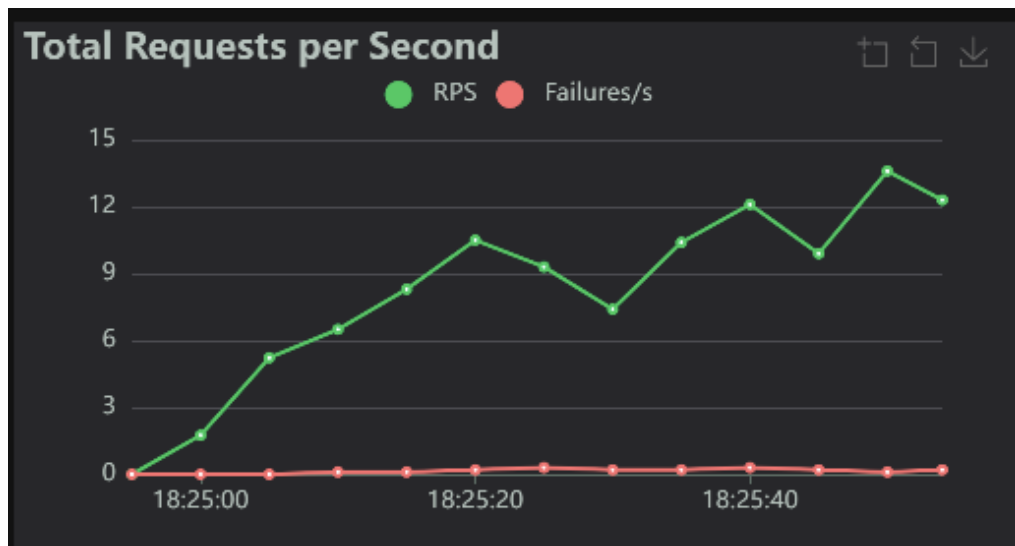


Рисунок 5.3 – график динамики RPS

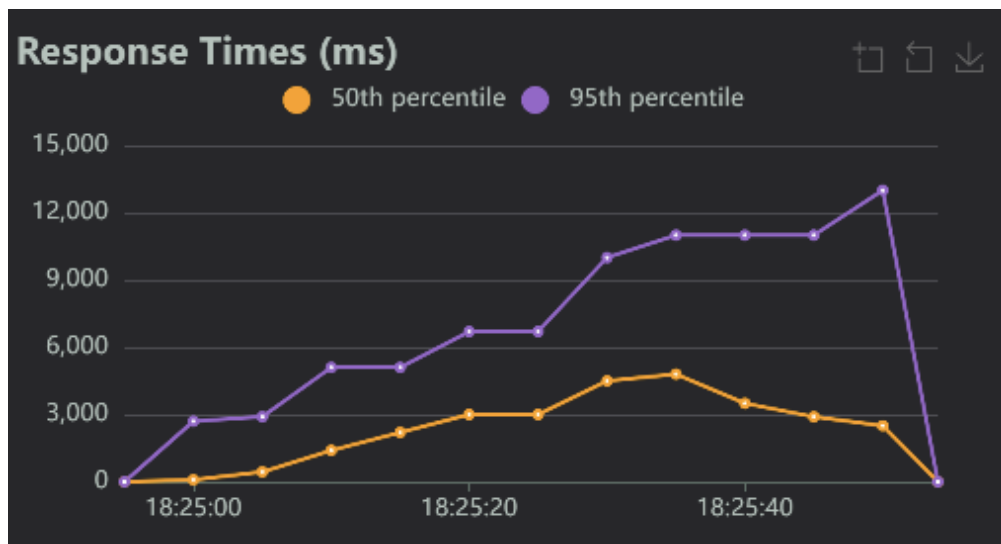


Рисунок 5.4 – график динамики времени ответа

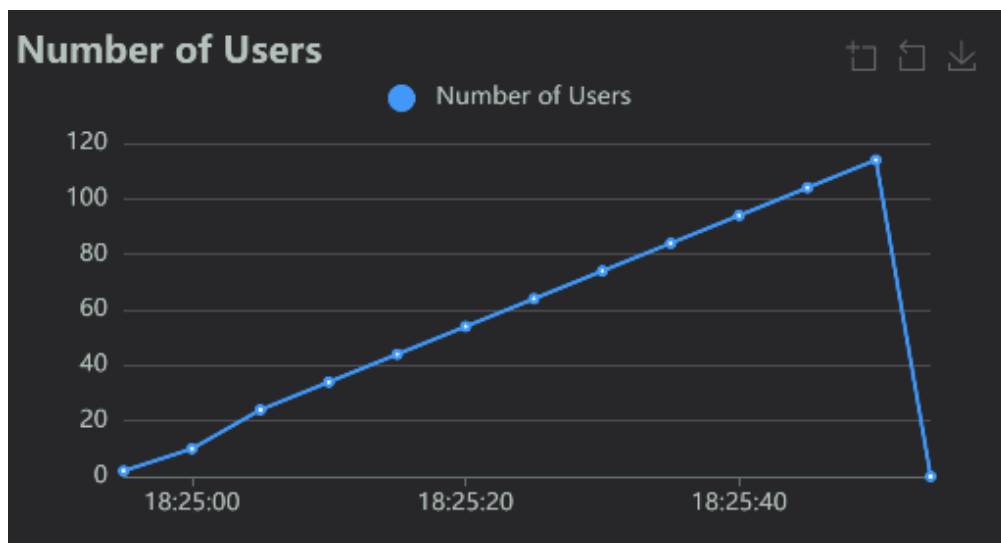


Рисунок 5.5 – график динамики количества пользователей

Сервер продемонстрировал способность обслуживать в среднем около 10 запросов в секунду при одновременной работе около 100 виртуальных пользователей. Медианное время отклика для смешанной нагрузки составило 2500 мс, однако на высоких значениях конкурентности наблюдался значительный рост задержек: 95-й перцентиль достигает 11000 мс, что свидетельствует о наличии «тяжёлых» операций, ограничивающих общую производительность.

Наименьшее время ответа зафиксировано для эндпоинта GET /auth/validate-access-token (медиана 480 мс), что объясняется его простотой — проверка JWT не требует обращения к базе данных. Напротив, наиболее ресурсоёмкими оказались операции POST /auth/register-init (медиана 6900 мс) и POST /auth/refresh (5400 мс). Вероятной причиной является выполнение криптографических операций (хеширование пароля, генерация новых токенов, шифрование данных), а также интенсивное взаимодействие с СУБД.

За время теста зафиксировано 10 ошибок (1,7% от общего числа запросов). Все они сконцентрированы на двух эндпоинтах: /auth/login-init (5 ошибок с кодом 403, что связано с превышением лимита попыток входа у части пользователей) и /auth/refresh (5 ошибок с нулевым HTTP-статусом, указывающих на таймауты соединений при пиковой нагрузке). Остальные эндпоинты отработали без сбоев.

Динамические графики подтверждают ожидаемую зависимость: с ростом числа пользователей увеличивается RPS, однако одновременно возрастает и время обработки запросов, что говорит о приближении к пределу масштабирования текущей конфигурации.

Таким образом, результаты нагрузочного тестирования позволили количественно оценить производительность системы, выявить наиболее медленные эндпоинты и зафиксировать характер ошибок, возникающих при превышении допустимой нагрузки. Полученные данные могут быть

использованы для последующей оптимизации серверного кода и планирования ресурсов при развёртывании системы в промышленной среде.

## 5.4 Комплексное тестирование

Комплексное тестирование является завершающим этапом проверки разработанной системы и проводится с целью подтверждения корректности реализации полного жизненного цикла аутентификации пользователя – от регистрации до выхода из системы – в условиях, максимально приближенных к реальной эксплуатации. В отличие от интеграционных тестов, проверяющих изолированные эндпоинты, комплексные сценарии воспроизводят длинные цепочки действий, включая ожидание истечения токенов и обработку ошибочных ситуаций.

Автоматизация комплексных сценариев выполнена на языке Python с использованием синхронной библиотеки `httpx` и модуля `threading` для параллельного исполнения. Разработано четыре сценария:

- 1) включает: регистрацию, валидацию access-токена, ожидание истечения access-токена, проверка отклонения истекшего токена, успешный выпуск нового access-токена через refresh, валидация нового токена, выход (отзыв refresh-токена), проверка невозможности повторного использования отозванного refresh-токена, инициализация входа с корректными данными;

- 2) аналогичен первому, но дополнительно после обновления access-токена ожидается истечение refresh-токена, затем фиксируется отказ обновления по истекшему refresh-токену и выполняется успешная инициализация входа;

- 3) повторяет регистрацию, валидацию, обновление, выход, после чего проверяет реакцию сервера на попытку входа с несуществующим идентификатором пользователя;

4) аналогичен третьему, но на последнем шаге проверяется попытка входа с некорректным паролем.

Все сценарии запускались одновременно в четырёх параллельных потоках на виртуальной лаборатории, описанной в разделе 5.1. Параметры токенов (1 минута для access, 2 минуты для refresh) соответствовали конфигурации, заданной в переменных окружения сервера на момент запуска тестирования. Результаты каждого шага (наименование, HTTP-статус, чистое время выполнения запроса и время искусственного ожидания) автоматически записывались в CSV-файл. На рисунке 5.6 – скриншот вывода консоли после проведения тестирования, а в таблице 5.2 – фрагмент данных CSV-файла результатов (исключены колонки времени, затраченного на сон и суммарного).

```
(.venv) PS C:\Inst\4_course\8_sem_2\NIR\server\app\tests\complex> python test_complex.py
Starting complex tests (parallel)...
=== Scenario 1 started ===
=== Scenario 2 started ===
=== Scenario 3 (wrong user_id) started ===
=== Scenario 4 (wrong password) started ===
Scenario 1: waiting 70 s ...
Scenario 2: waiting 70 s ...
Scenario 4 finished.
Scenario 3 finished.
Scenario 2: waiting 60 s ...
Scenario 1 finished.
Scenario 2 finished.
Sorting results by scenario...
All scenarios done. Results in results_complex.csv
(.venv) PS C:\Inst\4_course\8_sem_2\NIR\server\app\tests\complex>
```

Рисунок 5.6 – вывод консоли после завершения тестирования

Таблица 5.2 – Результаты комплексного тестирования

| № сценария | Шаг сценария        | Код | Время обработки запросов (мс) |
|------------|---------------------|-----|-------------------------------|
| 1          | register-init       | 201 | 843.921                       |
| 1          | register-accept     | 201 | 47.568                        |
| 1          | validate-a1         | 200 | 94.057                        |
| 1          | validate-a1-expired | 401 | 12.160                        |
| 1          | refresh             | 200 | 45.856                        |
| 1          | validate-a2         | 200 | 1.934                         |
| 1          | logout              | 204 | 34.528                        |
| 1          | refresh-revoked     | 401 | 3.045                         |
| 1          | login-init-valid    | 200 | 225.765                       |
| 1          | TOTAL SCENARIO 1    | OK  | 1308.834                      |

Окончание таблицы 5.2

| № сценария | Шаг сценария         | Код | Время обработки запросов (мс) |
|------------|----------------------|-----|-------------------------------|
| 2          | register-init        | 201 | 843.766                       |
| 2          | register-accept      | 201 | 106.868                       |
| 2          | validate-a1          | 200 | 36.214                        |
| 2          | validate-a1-expired  | 401 | 12.209                        |
| 2          | refresh-1            | 200 | 75.293                        |
| 2          | validate-a2          | 200 | 2.182                         |
| 2          | refresh-expired      | 401 | 15.074                        |
| 2          | login-init-valid     | 200 | 220.043                       |
| 2          | TOTAL_SCENARIO_2     | OK  | 1311.647                      |
| 3          | register-init        | 201 | 843.409                       |
| 3          | register-accept      | 201 | 136.307                       |
| 3          | validate-a1          | 200 | 6.373                         |
| 3          | refresh              | 200 | 65.851                        |
| 3          | validate-a2          | 200 | 3.605                         |
| 3          | logout               | 204 | 5.159                         |
| 3          | login-init-wrong-uid | 404 | 217.388                       |
| 3          | TOTAL_SCENARIO_3     | OK  | 1278.091                      |
| 4          | register-init        | 201 | 841.348                       |
| 4          | register-accept      | 201 | 76.765                        |
| 4          | validate-a1          | 200 | 64.588                        |
| 4          | refresh              | 200 | 36.973                        |
| 4          | validate-a2          | 200 | 32.994                        |
| 4          | logout               | 204 | 6.417                         |
| 4          | login-init-wrong-pw  | 401 | 217.174                       |
| 4          | TOTAL_SCENARIO_4     | OK  | 1276.260                      |

Все проверенные сценарии отработали успешно – зафиксированы ожидаемые HTTP-статусы на каждом шаге, ошибок не возникло. Это подтверждает корректность реализации серверной логики, включая обработку истечения токенов, отклонение невалидных учётных данных и защиту от повторного использования отозванных refresh-токенов.

Время ответов сервера оставалось стабильными на протяжении всего тестирования. Наиболее ресурсоёмким шагом, как и в предыдущих видах тестирования, ожидаемо оказалась инициализация регистрации (/auth/register-init) с медианным временем около 843 мс, что обусловлено выполнением криптографического хеширования пароля и записью в базу данных нового пользователя. Остальные операции (подтверждение регистрации, выпуск и

валидация токенов, выход) выполнялись за 2–140 мс. Обработка запросов с заранее известными некорректными параметрами (неверный пароль, несуществующий идентификатор, отозванный или истекший токен) занимала единицы миллисекунд, так как сервер прерывает проверку на ранних этапах, не доходя до ресурсоёмких вычислений.

Параллельное исполнение сценариев не оказало заметного негативного влияния на время отклика: колебания длительности однотипных запросов в разных потоках незначительны. Суммарное чистое процессорное время обработки всех запросов одного сценария составило приблизительно 1,3 секунды, в то время как общее время прогона с учётом принудительных ожиданий достигало 131 секунды для сценария 2, что полностью соответствует временным срокам годности токенов на момент проведения тестирования.

Таким образом, комплексное тестирование подтвердило, что разработанная система мультифакторной аутентификации корректно реализует весь жизненный цикл пользователя. Полученные метрики производительности согласуются с результатами интеграционного и нагрузочного тестирования и могут служить основой для дальнейшей оптимизации и масштабирования системы.

## **5.5 Итоги**

В рамках данной главы было выполнено тестирование и рефакторинг реализованной системы мультифакторной аутентификации. С использованием сконфигурированного виртуального лабораторного стенда проведены интеграционное, нагрузочное и комплексное тестирования.

Интеграционное тестирование подтвердило корректность реализации всех HTTP-эндпоинтов сервера аутентификации и их взаимодействия с базой данных. Все проверенные сценарии (успешные и ошибочные) отработали с

ожидаемыми статусами, а собранные метрики времени ответа продемонстрировали высокую производительность системы в условиях низкой нагрузки. В ходе первого прогона была выявлена и оперативно устранена ошибка, связанная с необработанным исключением при расшифровании некорректного ОТР, после чего сервер стал возвращать унифицированный ответ, не раскрывая внутреннюю логику потенциальному злоумышленнику.

Нагрузочное тестирование позволило количественно оценить пропускную способность системы. Сервер показал способность обслуживать в среднем около 10 запросов в секунду при одновременной работе около 100 виртуальных пользователей. Наиболее ресурсоёмкими операциями ожидаемо оказались инициализация регистрации и обновление токенов, что объясняется выполнением криптографических вычислений и интенсивным взаимодействием с СУБД. Наименьшее время ответа зафиксировано для облегчённого эндпоинта верификации access-токена, не требующего обращения к базе данных. Уровень ошибок на пиковой нагрузке составил 1,7%, все они сконцентрированы на двух эндпоинтах и обусловлены как логикой ограничения попыток входа, так и сетевыми таймаутами.

Комплексное тестирование подтвердило корректность реализации полного жизненного цикла пользователя, включая регистрацию, валидацию и обновление токенов, выход из системы, а также обработку истечения и отзыва refresh-токенов.

Таким образом, разработанная система мультифакторной аутентификации успешно прошла все запланированные виды тестирования. Выявленные узкие места могут быть учтены при дальнейшей оптимизации и масштабировании. Результаты тестирования подтверждают работоспособность и готовность системы к опытной эксплуатации, что позволяет констатировать выполнение очередной задачи выпускной квалификационной работы.

## ЗАКЛЮЧЕНИЕ

Целью выпускной квалификационной работы является обеспечение аутентичности пользователей информационных ресурсов вычислительной сети посредством мультифакторной аутентификации.

В рамках достижения цели были успешно выполнены все поставленные задачи:

- 1) исследована предметная область: рассматривались теоретические основы аутентификации и производился сравнительный анализ характеристик существующих популярных MFA-систем;
- 2) смоделированы объекты и процессы предметной области: анализировались модели систем аутентификации на основе различных факторов;
- 3) спроектирована система мультифакторной аутентификации: определены требования к системе, детально разработаны алгоритмы работы системы;
- 4) выполнена программная реализация спроектированной системы: описан процесс разворачивания и конфигурирования компонентов системы, согласно принципам слоистой архитектуре и ООП программно реализованы компоненты системы;
- 5) проведено тестирование и рефакторинг реализованной системы: описана конфигурация виртуальной лаборатории, на которой успешно пройдены три типа тестирования: интеграционное, нагрузочное и комплексное.

Практические результаты работы включают:

- сводную таблицу результатов сравнительного анализа характеристик существующих систем мультифакторной аутентификации;

- блок-схемы алгоритмов работы и UML-диаграммы последовательностей процессов моделей систем аутентификации;
- UML-диаграмма компонентов, BPMN-диаграмма основного сценария работы, ER-диаграмма базы данных и блок-схемы алгоритмов работы спроектированной системы мультифакторной аутентификации;
- сводная таблица технологического стека спроектированной системы мультифакторной аутентификации;
- UML-диаграммы компонентов и их классов разработанной системы мультифакторной аутентификации;
- сводные таблицы результатов произведенного тестирования разработанной системы мультифакторной аутентификации: интеграционного, нагрузочного и комплексного;
- исходный код компонентов системы;
- описание инфраструктуры виртуального лабораторного стенда с детальной инструкцией проведения автоматизированного тестирования разработанной системы мультифакторной аутентификации.

Все результаты работы в полном объеме загружены в публичный GitHub-репозиторий. Репозиторий содержит README-файлы с описанием файловой структуры проекта, а также инструкциями запуска компонентов разработанной системы и проведения ее тестирования.

В итоге, система мультифакторной аутентификации разработана и успешно протестирована, а значит цель работы по обеспечению аутентичности пользователей информационных ресурсов вычислительной сети посредством мультифакторной аутентификации можно считать достигнутой.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Афанасьев, А. А. Аутентификация. Теория и практика обеспечения безопасного доступа к информационным ресурсам: учебное пособие для вузов / А. А. Афанасьев, Л. Т. Веденьев, А. А. Воронцов и др.; под ред. А. А. Шелупанова, С. Л. Груздева, Ю. С. Нахаева. – 2-е изд., стереотип. – М.: Горячая линия–Телеком, 2012. – 550 с.
2. Чужиков, Н. О. Многофакторная аутентификация // Молодой ученый. – 2022. – № 21 (416). – С. 226–228.
3. Сидоркина, И. Г. Классификация методов аутентификации человека / Сидоркина И. Г., Канаев Р. В., Меркушев О. Ю. // Вестник ВУиТ. 2009. № 12. – С. 52–61.
4. Подгаев, А. Г. Проблемы безопасности систем двухфакторной аутентификации / А. Г. Подгаев, А. А. Подгаев // Ученые заметки ТОГУ. – 2019. – Т. 10. – № 4. – С. 256–260.
5. Юрьев Д. Р., Рогова О. С. Сравнительный анализ двухфакторной аутентификации – Владивосток: Технические науки – от теории к практике, 2017, № 6 – С. 46–51.
6. Попов, В. Г. Многофакторная аутентификация пользователя / Попов В. Г., Галиаскаров Д. Ф. // Научный электронный журнал меридиан. – 2021. – № 4 (57). – С. 237–239.
7. Стефанова, Н. А. Многофакторная аутентификация как инструмент информационной безопасности в цифровой экономике // Актуальные вопросы современной экономики. – 2020. – № 4. – С. 246–252.
8. Перевертун, Д. Р. Методы аутентификации и управления доступом // Международный журнал информационных технологий и энергоэффективности. – 2024. – № 5 (43). – С. 77–84.

9. Коломойцев, В. С. Формульный метод аутентификации пользователя / Коломойцев В. С., Марковский Е. С., Дубровина Д. А. // Инновационное приборостроение. – 2023. – № 3. – С. 77–83.
10. Корнев, А. С. Обеспечение безопасности систем аутентификации / Корнев А. С., Аминев А. Р. // Духовная ситуация времени. Россия XXI век. – 2023. – № 3 (32). – С. 1–2.
11. Рябцев, Д. К. Анализ современных методов хранения паролей // Безопасность цифровых технологий. – 2024. – № 1 (112). – С. 74–89.
12. Метельков, А. Н. О проблеме аутентификации с использованием паролей при информационном взаимодействии // Национальная безопасность и стратегическое планирование. – 2020. – № 3 (31). – С. 59–68.
13. Калининский, Д. С. Сравнительный анализ современных механизмов аутентификации в контексте информационной безопасности // Оригинальные исследования. – 2023. – № 6. – С. 81–89.
14. Надейкина, В. С. Анализ способов реализации системы многофакторной аутентификации / Надейкина В. С., Лагуткина Т. В. // Научный результат. Информационные технологии. – 2022. – № 4. – С. 59–66.
15. Подшибякин, А. С. Результаты анализа функционирования механизмов защиты в беспроводных сетях передачи данных / Подшибякин А. С., Ушаков И. А., Шинкаренко А. Ф. // Труды Военно-Космической Академии имени А.Ф. Можайского. – 2021. – № 678. – С. 163–174.
16. Глотов, А. И. Разработка усовершенствованных генераторов паролей / Глотов А. И., Котилевец И. Д., Иванова И. А. // Ученые записки УЛГУ. Серия: математика и информационные технологии. – 2021. – № 1. – С. 13–21.

17. Докшин, А. Д. Исследование подходов для аутентификации пользователей беспроводной сети с применением различных LDAP решений / Докшин А. Д., Ковцур М. М., Прудников С. В., Таргонская А. И. // Научные технологии в космических исследованиях Земли. – 2021. – № 3. – С. 28–35.
18. Таныгин, М. О. Установление доверительного канала обмена данными между источником и приёмником информации с помощью модифицированного метода одноразовых паролей / Таныгин М. О., Алшаи Х. Я., Алтухова В. А., Марухленко А. Л. // Известия Юго-Западного Государственного Университета. Серия: управление, вычислительная техника, информатика. Медицинское приборостроение. – 2018. – № 4 (29). – С. 63–71.
19. Юркин, Д. В. Формализованный анализ протоколов аутентификации / Юркин Д. В., Уткина А. А., Первушин А. О. // Информационно-управляющие системы. – 2018. № 2 (93). – С. 76–83.
20. Богданов, Д. С. Классификация и сравнительный анализ технологий многофакторной аутентификации в веб-приложениях / Богданов Д. С., Ключев С. Г. // Моделирование, оптимизация и информационные технологии. – 2020. – № 1 (28). – Порядковый номер 17.
21. Никешин, А. В. Обзор расширяемого протокола аутентификации и его методов / Никешин А. В., Шнитман В. З. // Труды института системного программирования РАН. – 2018. – № 2. С. 113–148.
22. Стенин, С. В. Анализ методов аутентификации по используемым средствам / Стенин С. В., Рак Д. В. // Альманах Пермского Военного Института войск национальной гвардии. – 2021. – № 4 (4). – С. 362–366.
23. Василенко, К. А. Корпоративные вычислительные сети: процедуры аутентификации и идентификации / Василенко К. А., Золкин А. Л., Абрамов Н. В., Курганов Д. О. // Инфокоммуникационные технологии. – 2020. – № 1. – С. 61–67.

24. Басараб, М. А. Алгоритм аутентификации и формирования разовых ключей в имитаторе лабораторного стенда объединенной сети ПД специального назначения / Басараб М. А., Бельфер Р. А., Кравцов А. В., Никулина Т. // Первая миля. – 2019. – № 2 (79). – С. 62–69.

25. Обзор рынка российских инструментов многофакторной аутентификации // Cyber Media [Электронный ресурс]. – 2022. – URL: <https://securitymedia.org/analytics/obzor-rynka-rossijskih-instrumentov-mnogofaktornoj-autentifikacii.html> (дата обращения: 24.06.2024).

26. Краткое описание нотации BPMN // Хабр [Электронный ресурс]. – 2022. – URL: <https://habr.com/ru/companies/auriga/articles/667084/> (дата обращения: 25.06.2024).

27. UML: обзор основных типов диаграмм, диаграмма компонентов. Часть 2 // Хабр [Электронный ресурс]. – 2023. – URL: <https://habr.com/ru/articles/756552/> (дата обращения: 26.06.2024).

28. Простое руководство по диаграммы компонентов // Creately Blog [Электронный ресурс]. – 2022. – URL: <https://creately.com/blog/ru/uncategorized-ru/учебное-пособие-по-компонентной-диаг/> (дата обращения: 26.06.2024).

29. Authentication Using Fingerprint in Android // GeeksforGeeks [Электронный ресурс]. – 2021. – URL: [https://translated.turbopages.org/proxy\\_u/en-ru.ru.fd6c520a-667c2836-71bb5317-74722d776562/https/www.geeksforgeeks.org/authentication-using-fingerprint-in-android/](https://translated.turbopages.org/proxy_u/en-ru.ru.fd6c520a-667c2836-71bb5317-74722d776562/https/www.geeksforgeeks.org/authentication-using-fingerprint-in-android/) (Дата обращения: 26.06.2024).

30. Android Fingerprint API: приделываем аутентификацию по отпечатку // Хабр [Электронный ресурс]. – 2016. – URL: <https://habr.com/ru/companies/e-legion/articles/317706/> (Дата обращения: 26.06.2024).

31. Как выбрать язык программирования для создания Android-приложения // Хабр [Электронный ресурс]. – 2019. – URL: <https://habr.com/ru/articles/477578/> (Дата обращения: 26.06.2024).

32. Использование диаграммы классов UML при проектировании и документировании программного обеспечения // Хабр [Электронный ресурс]. – 2021. – URL: <https://habr.com/ru/articles/572234/> (Дата обращения: 27.06.2024).

33. Утечки данных в России // TAdviser [Электронный ресурс]. – 2025. – URL: [https://www.tadviser.ru/index.php/Статья:Утечки\\_данных\\_в\\_России](https://www.tadviser.ru/index.php/Статья:Утечки_данных_в_России) (дата обращения: 29.01.2024).

34. Экспертное мнение: перспективы российского рынка многофакторной аутентификации // IT Channel News. – 2024. – 24 июня. – URL: <https://www.novostiitkanala.ru/news/detail.php?ID=177932> (дата обращения: 29.01.2025).

35. Which Database You Should Learn in 2025 // Geeks for Geeks [Электронный ресурс]. – 2025. – URL: <https://www.geeksforgeeks.org/which-database-you-should-learn/> (дата обращения: 05.02.2025).

36. Comparison Between MongoDB vs MySQL vs PostgreSQL // Linearloop [Электронный ресурс]. – 2024. – URL: <https://www.linearloop.io/blog/comparison-between-mongodb-vs-mysql-vs-postgresql> (дата обращения: 05.02.2025).

37. Database Comparison - SQL vs. NoSQL (MySQL vs PostgreSQL vs Redis vs MongoDB) // Profil [Электронный ресурс]. – 2019. – URL: <https://profil-software.com/blog/development/database-comparison-sql-vs-nosql-mysql-vs-postgresql-vs-redis-vs-mongodb/> (дата обращения: 06.02.2025).

## **ПРИЛОЖЕНИЕ А**

### **Ссылка на разработанное решение**

<https://github.com/KATEHOK/NIR-spring-2026>



## **ПРИЛОЖЕНИЕ Б**

**Ссылка на видеодемонстрацию работы разработанного решения**

<https://www.youtube.com/watch?v=IajmY2cxHq0>



## ПРИЛОЖЕНИЕ В

### Файл Locust\_...\_requests.csv

Type,Name,Request Count,Failure Count,Median Response Time,Average Response Time,Min Response Time,Max Response Time,Average Content Size,Requests/s,Failures/s,50%,66%,75%,80%,90%,95%,98%,99%,99.9%,99.99%,100%

POST,/auth/login-

accept,32,0,4100,3756.0521968734975,8.26140004210174,8731.358299963176,26.0,0.5352599764465672,0.0,4100,4700,5600,7700,8500,8600,8700,8700,8700,8700,8700,8700

POST,/auth/login-

init,37,5,3500,4223.238956746393,232.3190999450162,18995.567700010724,143.35135135135135,0.6188943477663433,0.08363437131977612,3500,4900,5700,6400,8000,10000,19000,19000,19000,19000,19000,19000

DELETE,/auth/logout,28,0,2200,3834.577046422055,6.003499962389469,16272.12670003064,0.0,0.46835247939074626,0.0,2300,4200,7000,7500,9000,12000,16000,16000,16000,16000,16000

POST,/auth/refresh,36,5,5400,5372.3256472342955,34.279100014828146,13958.043200080283,493.4166666666667,0.6021674735023881,0.08363437131977612,5400,7600,9500,9600,12000,13000,14000,14000,14000,14000,14000

POST,/auth/register-

accept,121,0,2900,4159.27401982625,38.75309997238219,15128.186200046912,573.0,2.023951785938582,0.0,2900,4000,5500,7300,9900,11000,14000,14000,15000,15000,15000

POST,/auth/register-

init,142,0,6900,7399.174423946533,2454.624700010754,19406.82860009838,103.0,2.375216145481642,0.0,7100,9600,10000,10000,11000,13000,16000,19000,19000,19000,19000

GET,/auth/validate-access-

token,188,0,480,562.6748941505596,2.606600057333708,2125.032900017686,29  
.0,3.144652361623582,0.0,480,710,830,910,1200,1300,1700,2100,2100,210  
0  
,Aggregated,584,10,2500,3830.4164332183386,2.606600057333708,19406.82860  
009838,194.02397260273972,9.768494570149851,0.16726874263955224,2500,4  
700,6200,7500,10000,11000,13000,15000,19000,19000,19000

## ПРИЛОЖЕНИЕ Г

### Файл Locust\_...\_failures.csv

Method,Name,Error,Occurrences

POST,/auth/refresh,CatchResponseError('Refresh failed: 0'),5

POST,/auth/login-init,CatchResponseError('Login init failed: 403'),5